

情報工学実験Ⅲ・実験報告書

実験テーマ

実験題目

C 前半

ディープラーニングの実験

Experiment of deep learning

班番号

■予習自己点検（予習で理解してきた項目に自身で○を付ける）

はじめに・全用語・目的とねらい・実験手順・結果と考察事項

学生番号	氏名
------	----

共同実験者（実験データを共有する者）

実験日（１） 年 月 日（ ）	学生番号	学生番号	学生番号	学生番号
	氏名	氏名	氏名	氏名
実験日（２） 年 月 日（ ）	学生番号	学生番号	学生番号	学生番号
	氏名	氏名	氏名	氏名
実験日（３） 年 月 日（ ）	学生番号	学生番号	学生番号	学生番号
	氏名	氏名	氏名	氏名

予習レポート 年 月 日（ ）	本レポート 初回 年 月 日（ ）	本レポート ２回目 年 月 日（ ）	本レポート ３回目 年 月 日（ ）
■禁止 簡易な図のコピー カラー印刷・文章丸写し 未完成 公的機関以外の参照 ■不足[全般] レイアウト・フォント 見出し・文章 図・表・式 項番・ページ番号 ■誤り又は過不足[項目] 表紙・目的・原理・方法 予習課題・参考文献	■禁止 簡易な図のコピー カラー印刷・文章丸写し 未修正・未完成 公的機関以外の参照 指摘なし ■不足[全般] レイアウト・フォント 見出し・文章 図・表・式 項番・ページ番号 指摘なし ■誤り又は過不足[項目] 表紙・目的・原理・方法 予習課題・使用器具 実験結果・考察課題 実験考察・参考文献 指摘なし	■禁止 簡易な図のコピー カラー印刷・文章丸写し 未修正・未完成 公的機関以外の参照 指摘なし ■不足[全般] レイアウト・フォント 見出し・文章 図・表・式 項番・ページ番号 指摘なし ■誤り又は過不足[項目] 表紙・目的・原理・方法 予習課題・使用器具 実験結果・考察課題 実験考察・参考文献 指摘なし	

1. 目的

Python を用いたニューラルネットワークの構築を通して、Python の記述方法およびディープラーニングの基本原則を理解することが全体の目的である。C1 実験においては、ペアプログラミングで Python コーディングを行い、ソフトウェアの共同開発手法の基礎およびディープラーニング実装に必要なコードを作成することを目的とする。また、C2 実験においてはディープニューラルネットワークの基本的な構造を実装し、理解することを目的とする。

2. 原理

2.1 ペアプログラミング

ソフトウェア開発手法の一種で、2 人のプログラマーが 1 台の PC を操作してプログラムを行うことを指す。アジャイル手法の一種である XP 手法で推奨される。

2.2 機械学習

機械学習 (Machine Learning) とは、人工知能を実現するためのアルゴリズムの集合の一種である。主に教師あり学習、教師なし学習、強化学習に分けられる。教師あり学習では学習データ及び正解データを学習に用いる。例えば、犬や猫の画像を学習データとし、それぞれの画像を「犬」や「猫」のラベルに割り当てる。このラベルを正解データという。本実験で扱うディープラーニングは、教師あり学習である。

2.3 ニューラルネットワーク

脳内の神経細胞 (ニューロン) のネットワーク構造を模して作られた数学モデルである。入力層、中間層、出力層によりネットワークが構成される。画像データなどを次元の数値データへ変換することで、入力とする。入力されたデータはネットワーク上のノード結合を通して計算され、最終的に出力となる。多くの場合、出力は画像の種類のそれぞれの確率のベクトルとして表される。ニューラルネットワークをランダムに構築しても、正しく画像認識などはなされない。入力を多数用意し、出力結果が正しいか正しくないかを評価し、そのフィードバックを用いてニューラルネットワークの結合強度を変えていく過程を学習と呼ぶ。学習により精度が十分高くなったのち、ニューラルネットワークのテストを行う。テストでは学習に用いなかったデータである、テストデータを入力として用いる。テストデータに対する精度が十分に高い場合、学習は成功したと判定する。

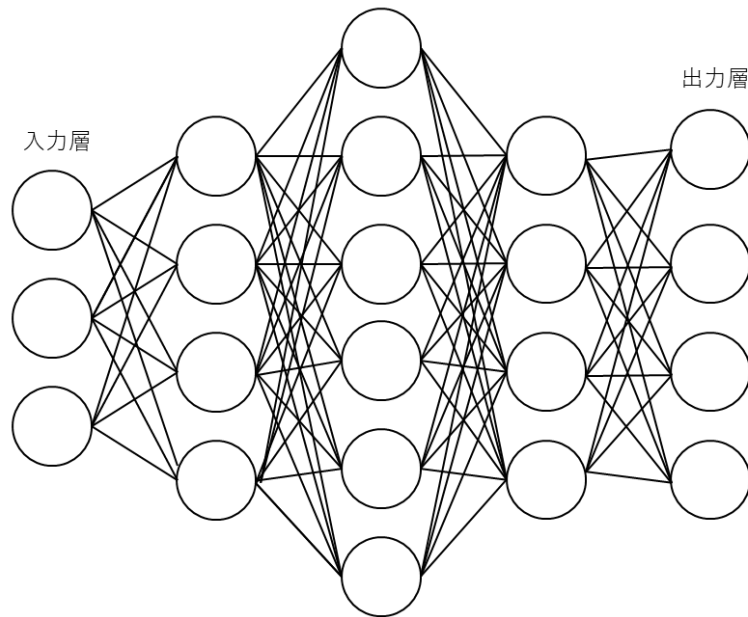


図 1 ディープニューラルネットワークの例

2.4 ディープラーニング

ニューラルネットワークの中間層が複数ある、深いニューラルネットワークを用いた機械学習である。

図 1 に示した例は、中間層が 3 層あるディープニューラルネットワークの例となる。実際のディープニューラルネットワークでは、数千、数万以上の中間層を持つネットワークを用いることもある。図 1 中の丸い図形を、ニューロンやノードと呼ぶ。ノードどうしを結ぶ線はシナプス、あるいはエッジと呼ぶ。この例では、隣り合った層のノードがすべて結合されている。このような結合を全結合と呼ぶ。この例では、入力として 4 個のデータを使い、ノードごとに割り当てられた「バイアス値」およびそれぞれのエッジに割り当てられた「重み値」をもとに、入力層、中間層を経て、最終的な計算結果を出力層に出力する。

2.5 分類問題と回帰問題

教師あり学習が対象とする問題は主に、分類問題と回帰問題である。分類問題とは、例えば与えられた画像が、「犬」か「猫」。あるいは「猿」のうち、どの画像が分類する問題をいう。回帰問題とは、例えば不動産の価格を「駅からの距離」「大きさ」「築年数」などの値を入力とした場合に、不動産の推定価格の出力を行う問題をいう。本実験では、分類問題を扱う。図 2 に分類問題の例を示す。

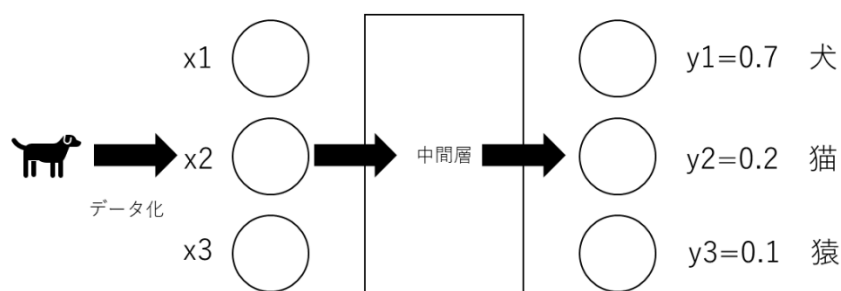


図 2 分類問題の例

画像をデータ化し、入力ベクトル (x_1, x_2, x_3) を生成する。それらを、中間層を通してニューラルネットワーク上で伝搬し、出力をする。分類問題では、出力ノードの和を 1 とする場合が多い。それぞれのノードに画像のクラスを割り当て、値が最も高い（確率が最も高い）ノードへ割り当てられた画像のクラスが出力となる。この例では、3 個の出力ノード (y_1, y_2, y_3) にそれぞれ犬、猫、猿のクラスを割り当てた。その結果、入力画像が犬であると推定された。

2.6 活性化関数

ニューラルネットワークにおける計算は次のように行われる。あるノード n に入ってくる前の層のノードの値に、両者間のエッジに割り当てられている重みを乗算する。この演算をすべての入ってくる前の層のノードに対して行い、その和を求める。求めた和に、ノード n に割り当てられているバイアス値を加算する。この演算を入力層、中間層、出力層を通して行う。これらの演算は線形計算であり、いくら中間層をふやしても、それと等価な 1 層の中間層に変換できてしまう。そのため、ディープなニューラルネットワークを構築する意味がない。よって、それぞれのノードの演算結果を非線形な関数にかけて、そのノードの出力とする。この非線形な関数は活性化関数と呼ばれる。入力層、中間層においては、ステップ関数、シグモイド関数、ReLU 関数と呼ばれる活性化関数が多く用いられる。また、出力層においては分類問題の場合、ソフトマックス関数と呼ばれる関数が使われることが多い。

2.7 Google Colaboratory

Google Research が提供する、「Colaboratory」（略称：Colab）は、誰でもブラウザ上で Python の記述、実行ができるというサービスである。「環境構築が不要」「GPU への無料アクセス」「簡単に共有」という特徴を持ち、ウェブ上でコードを記述して実行できる、インタラクティブな Python 開発環境である。Colab ノートブックという形式をとっており、大きく「コードセル」と「テキストセル」が存在する。これにより、実行可能なコードと、画像などを含むリッチテキストを一つのドキュメントで記述可能である。

「コードセル」には Python コードや Linux のシステムコマンドを記述でき、セル単位でコードを実行することができる。Python コードの場合はそのまま記述可能だが、システムコマンドの場合は先頭に `!` を付ける必要があり、例えばファイルを一覧表示する `ls` コマンドは `!ls` のように記述する。「テキストセル」にはマークダウン記法を用いて文章を記述可能であり、見出しや図・表の挿入、LaTeX を使った数式の追加も行える。

機械学習、ディープラーニング、画像処理、画像認識などを行う上で必要最低限なライブラリが予めインストールされた環境をすぐに利用できるため、「環境構築が不要」である。Google のサービスであるためメールアドレスを利用して Colab ノートブックを「簡単に共有」することができ、共同編集が容易。

Colab ノートブックは、最大存続期間が 12 時間の仮想マシンに接続することで実行可能。長時間（数十分～1 時間程度）放置すると、12 時間経過せずとも仮想マシンから切断され、ノートブック内の変数などが初期化される。GPU や TPU を搭載した仮想マシンも利用することができるため、「GPU への無料アクセス」が可能であるが、Colaboratory では状況に応じて変動する動的な使用制限が設けられている。リソースの保証を行わないことにより、無料でのリソース提供が実現されている。よって、Colaboratory を長時間に使用し続けるユーザー、直近のリソース使用量が多いユーザーは GPU と TPU

へのアクセスが一時的に制限される場合がある。

Colaboratory (<http://colab.research.google.com/?hl=ja>) を開き、左上の「ノートブックを新規作成」を選択することでコーディング環境を作成できる。

2.8 Python

プログラミング言語の一つであり、機械学習に用いられるライブラリが充実していることから、機械学習、ディープラーニングの実装に広く用いられる。Python は 1990 年に登場した比較的新しい言語であるため、オブジェクト指向や関数型言語の様々な特徴が盛り込まれている。以下に簡単な説明を示す。

① 算術演算

加算や乗算などは図 3 のように行える。1・2 や 4*5 の左にある 1 は、行数である。[1]、[2]などはコードの実行順である。

[1]	1 1-2
▼	-1
[2]	1 4*5
✓ 0 秒	20
[3]	1 3**4
✓ 0 秒	81

図 3 加算、乗算などの演算

② データ型

Python には C や Java と同様なデータ型がある。type()関数によりデータの型が表示できる。図 4 に示す。

[4]	1 type(10)
✓ 0 秒	... int
[5]	1 type(2.718)
✓ 0 秒	float
[6]	1 type("hello")
✓ 0 秒	str

図 4 データ型の表示

③ 変数

x や **y** などのアルファベットを利用して変数 (Variable) を定義することができる。また、変数を使った計算や、変数への別の値の代入 (値の参照) が行える。Python の **print** 文は C 言語などと違い、型指定なしでも表示することが可能である。図 5 に示す。

[7] ✓ 0 秒	1 x=10 2 print(x)	10
[8] ✓ 0 秒	1 x=100 2 print(x)	100
[9] ✓ 0 秒	1 y=3.14 2 x*y	314.0
[10] ✓ 0 秒	1 type(x*y)	float

図 5 Python の変数による計算

④ リスト

単一の数値だけでなく、リスト (配列) としてデータをまとめられる。リストの要素は図 6 のように様々な方法でアクセス可能である。図 6 に示す。

2] 0 秒	1 a=[1, 2, 3, 4, 5] 2 print(a)	[37] ✓ 0 秒	1 a[0:2]
	... [1, 2, 3, 4, 5]		[1, 2]
3] 0 秒	1 a[0]	[38] ✓ 0 秒	1 a[1:]
	1		[2, 3, 4, 99]
4] 0 秒	1 a[4]	[39] ✓ 0 秒	1 a[3]
	5		[1, 2, 3]
5] 0 秒	1 a[4]=99	[40] ✓ 0 秒	1 a[-1]
	1 print(a)		[1, 2, 3, 4]
6] 0 秒	[1, 2, 3, 4, 99]	[41] ✓ 0 秒	1 a[:-2]
			[1, 2, 3]

図 6 リスト (配列) の作成および要素へのアクセス方法

⑤ ディクショナリ

キーと値をペアとしてデータを格納することができる。図 7 に示す。

[42] ✓ 0 秒	1 me={'height':180}
[43] ✓ 0 秒 ▼	1 me['height'] 180
[44] ✓ 0 秒 ▼	1 me['weight']=70 2 print(me) {'height': 180, 'weight': 70}

図 7 ディクショナリの使用例

⑥ ブーリアン (Boolean)

Python には bool という型がある。これは True,False のどちらかとなる。Bool 型に対する演算子として and,or,not がある。図 8 に示す。

[45] ✓ 0 秒	1 hungry=True
[46] ✓ 0 秒	1 sleepy=False
[47] ✓ 0 秒 ▼	1 type(hungry) bool
[48] ✓ 0 秒 ▼	1 not hungry False
[49] ✓ 0 秒 ▼	1 hungry and sleepy False
[50] ✓ 0 秒 ▼	1 hungry or sleepy True

図 8 ブーリアンの使用例

⑦ If 文

条件に応じて処理を分岐するには if や else を用いる。図 9 に示す。

[56] ✓ 0 秒	<pre>1 hungry=True 2 if hungry: 3 print("i'm hungry")</pre>
▼	i'm hungry
[58] ✓ 0 秒	<pre>1 hungry=False 2 if hungry: 3 print("i'm hungry") 4 else: 5 print("i'm not hungry") 6 print("i'm sleepy")</pre>
▼	i'm not hungry i'm sleepy

図 9 if 文の例

Python では、空白文字が重要な意味を持つ。この if 文においても if hungry: の次の文頭は 2 文字の空白がある。これはインデントであり、先の条件 (if hungry) が満たされると実行されるコードとなる。C 言語や Java と異なり、Python ではインデントでプログラムの階層を表す。

⑧ for 文

ループ処理を行うには for 文を用いる。図 10 に示す。

[59] ✓ 0 秒	<pre>1 for i in [1,2,3]: 2 print(i)</pre>
▼	1 2 3

図 10 for 文の例

ここでは、[1,2,3]というリストの中の要素を出力する例を示している。For...in...:という構文を用いると、リストなどのデータ集合の各要素へ順にアクセスが可能である。また、rangeを用いることにより、任意回数のループが作成可能である。図 11 に示す。

[60] ✓ 0 秒	1 for i in range(5): 2 print(i)
▼	0 1 2 3 4

図 11 range を使ったループの例

⑨ 関数

まとまりのある処理を関数 (function) として定義することができる。関数は C 言語や Java と同様に引数を取ることができる。図 12 に示す。

[61] ✓ 0 秒	1 def hello(): 2 print("hello, world!") 3 hello()
▼	hello, world!
[62] ✓ 0 秒	1 def hello(object): 2 print("Hello "+object+"!") 3 hello("cat")
▼	Hello cat!

図 12 関数の定義と実行の例

⑩ クラス

Int や str などのデータ型は組み込みのデータ型と呼ばれる、はじめから Python に組み込まれているデータ型である。ここで、新しいクラスを定義する。ユーザーが自身でクラスを定義すると、独自にデータ型を作成することができる。また、ユーザー定義のメソッド（クラス用の関数）や属性を定義することが可能となる。Python では、class というキーワードを用いてクラスを定義する。クラスは次のフォーマットに従い作成される。

class クラス名:

```
def __init__(self,引数,...): #コンストラクタ
...
def メソッド名 1(self,引数,...): #メソッド 1
...
def メソッド名 2(self,引数,...): #メソッド 2
...
```

ここで、__init__という特別なメソッドがあるが、初期化を行うためのメソッドである。この初期化用メソッドはコンストラクタ（constructor）とも呼ばれ、クラスのインスタンスが作成されるときに一度のみ呼ばれる。また、Python ではメソッドの第一引数に自分自身（自身のインスタンス）を表す self を明示的に書く。図 13 に示す。



```
[65]
✓ 0 秒
1 class Man:
2     def __init__(self,name):
3         self.name=name
4         print("Initialized!")
5     def hello(self):
6         print("Hello "+self.name+"!")
7     def goodbye(self):
8         print("Good-bye"+self.name+"!")

[66]
✓ 0 秒
1 m=Man("David")
2 m.hello()
3 m.goodbye()

✓
Initialized!
Hello David!
Good-byeDavid!
```

図 13 クラスの定義と使用例

⑪ NumPy

ディープラーニングの実装においては、配列や行列の計算が多く登場する。NumPy は Python のライブラリの一つで、それらの計算に有用なメソッドが含まれているためディープラーニングの実装に広く用いられる。

(ア) NumPy のインポート

NumPy のインポートは以下で実行される。

```
Import numpy as np
```

これを実行以降、NumPy に関するメソッドは `as` により `np` として参照可能となる。

(イ) NumPy 配列の生成

NumPy の配列を作成するために、`np.array()`メソッドを用いる。図 14 に示す。

[68]	1 x=np.array([1.0,2.0,3.0]) 2 print(x) 3 type(x)
✓ 0 秒	
▼	[1. 2. 3.] numpy.ndarray

図 14 NumPy 配列の生成の例

(ウ) NumPy の算術計算

NumPy 配列の算術計算の例を示す。これらの算術は、`x` と `y` が同じ型、次元、要素（要素数 3 の一次元配列）である必要がある。図 15 に示す。

[70]	1 x=np.array([1.0,2.0,3.0]) 2 y=np.array([2.0,4.0,6.0]) 3 x+y
✓ 0 秒	
▼	array([3., 6., 9.])
[71]	1 x-y
✓ 0 秒	
▼	array([-1., -2., -3.])
[72]	1 x/y
✓ 0 秒	
▼	array([0.5, 0.5, 0.5])
[73]	1 x*y
✓ 0 秒	
▼	array([2., 8., 18.])

図 15 NumPy の算術計算の例

(エ) NumPy の N 次元配列

NumPy では多次元の配列も作成することができる。例えば、2 次元配列は次のように作成可能である。図 16 に示す。

[74] ✓ 0 秒	1 A=np.array([[1, 2], [3, 4]]) 2 print(A)
▼	[[1 2] [3 4]]
[75] ✓ 0 秒	1 A.shape
▼	(2, 2)
[76] ✓ 0 秒	1 A.dtype
▼	dtype('int64')
[77] ✓ 0 秒	1 B=np.array([[3, 0], [0, 6]]) 2 A+B
▼	array([[4, 2], [3, 10]])
[78] ✓ 0 秒	1 A*B
▼	array([[3, 0], [0, 24]])
[79] ✓ 0 秒	1 A*10
▼	array([[10, 20], [30, 40]])

図 16 NumPy の 2 次元配列の例

A は 2×2 の行列である。shape と dtype で行列 A の形状とデータ型を参照できる。また、array を利用した行列の算術計算ができる。

(オ) ブロードキャスト

NumPy では形状の異なる配列の演算も可能である。先程の例では、A という 2×2 の行列に対する 10 というスカラ値の掛け算を行った。NumPy ではこのような演算を、10 というスカラ値を 2×2 の要素に拡大して行う。この機能をブロードキャストと呼ぶ。ブロードキャストの別の例としては、図 17 の計算では [1,2] と [3,4] の 2×2 の行列に対して [10,20] の 1×2 の行列を 2×2 の要素に拡大して行われた。

[80]	1 A=np.array([[1, 2], [3, 4]])
✓ 0 秒	2 B=np.array([10, 20])
	3 A*B
▼	array([[10, 40], [30, 80]])

図 17 ブロードキャストの例

(カ) 要素へのアクセス

NumPy 行列の各要素へのアクセスは次のように行う。図 18 に示す。

[81]	1 X=np.array([[51, 55], [14, 19], [0, 4]])
✓ 0 秒	2 print(X)
▼	[[51 55] [14 19] [0 4]]
[82]	1 X[0]
✓ 0 秒	array([51, 55])
[83]	1 X[0][1]
✓ 0 秒	np.int64(55)

図 18 NumPy 行列の要素へのアクセスの例

NumPy はこれまで見てきたインデックス操作に加え、配列によって各要素にアクセスすることもできる。このことを応用し、ある条件を満たす要素だけの取得なども可能である。図 19 に示す。

[84] ✓ 0 秒	1 X=X.flatten() 2 print(X)	[51 55 14 19 0 4]
[86] ✓ 0 秒	1 X[np.array([0,2,4])]	array([51, 14, 0])
[87] ✓ 0 秒	1 X[X>15]	array([51, 55, 19])

図 19 配列による要素へのアクセス

⑫ Matplotlib

ディープラーニングの実験においては、グラフの描写やデータの可視化が重要となる。Matplotlib はグラフ描画のためのライブラリである。ここでは Matplotlib の使用方法を簡単に示す。

(ア) 簡単なグラフの描写

グラフの描画は matplotlib の pyplot というモジュールを使用する。図 20 は sin 関数を描画する例となる。Colaboratory では pyplot を使ったコードを実行すると、グラフが表示できる。図 20 に示す。

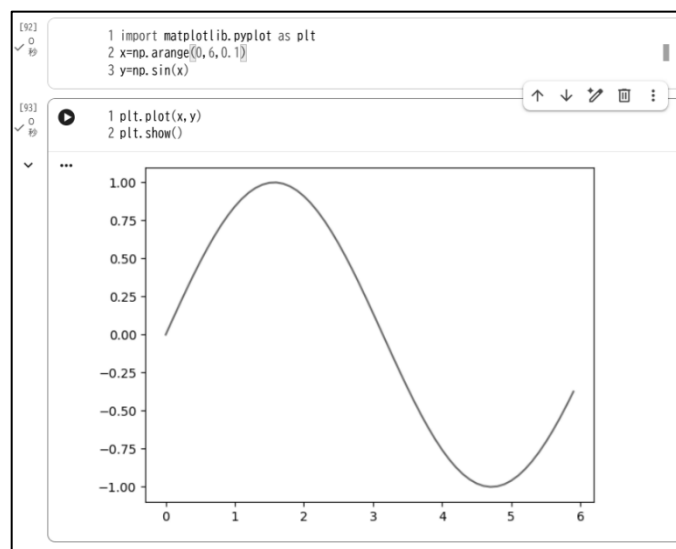


図 20 pyplot による sin 関数の描画

(イ) pyplot の機能

pyplot では一つの図への複数のグラフの描画、タイトルや x,y 軸のラベルの描画など
もできる。図 21 に cos 関数も追加し、ラベルを描画した例を示す。

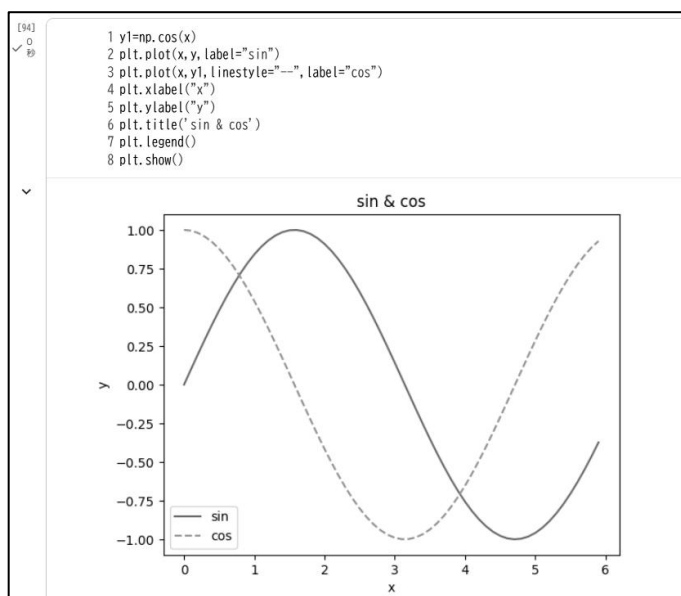


図 21 cos 関数、ラベルの追加

2.9 ミニバッチ入力

データを一個ずつニューラルネットワークに入力し、入力を得る際に時間がかかる場合がある。ミニバッチ入力とは、複数のデータをひとまとめにしてニューラルネットワークへの入力とすることである。ニューラルネットワークで出力を得る計算はすべて行列で記述できる。そのため、ミニバッチ化とは、入力を一次元のベクトルから複数の一次元ベクトルをまとめた 2 次元行列にすることに相当している。

2.10 データのフラット化

画像データは通常、2 次元データとして表される。機械学習のモデルの入力とするときに 2 次元データを一次元データに変換することがある。このことをデータのフラット化という。

2.11 One-Hot エンコーディング

例えば手書きで書かれた 1 文字の数字を認識する場合、正解ラベルは 0 から 9 までをとりうる。それらの正解ラベルとディープニューラルネットワークの出力を比較するときなどの演算を容易にするため、0 と 1 を要素に持つベクトルへ変換する。変換の例としては、5 は 00000100000 へ変換される。左から 6 番目の要素のみ 1 が立っており、ほかの数字についても同様の変換方式となる。この変換を、One-Hot エンコーディングと呼ぶ。

2.12 MNIST データベース

MNIST (Modified Nation Institute of Standards and Technology) データベースは、手書き数字画像の大規模データベースである。60,000 枚の学習用画像と 10,000 枚のテスト用画像を含んでいる。ディープラーニングのモデルの評価に広く用いられている。

3. 方法

3.1 C1: ペアプログラミングによる Python コーディング

- (1) Google Colaboratory を実験班の PC 上で立ち上げ、自分の班のノートブックを作成した。
- (2) 自分の班のノートブックを班員全員で共有可能にした。
- (3) 配布される課題にペアプログラミングにより回答し、自分の班のノートブックから実行を確認した。

3.2 C2: Python によるニューラルネットワークの構築

- (1) 図 22 のニューラルネットワークについて NumPy を用いて実装し、出力を求めた。入力は $x_1, x_2 = (0, 1)$ である。それぞれのノードに書かれている数字はバイアスであり、それぞれの辺に書かれている数字は重みである。

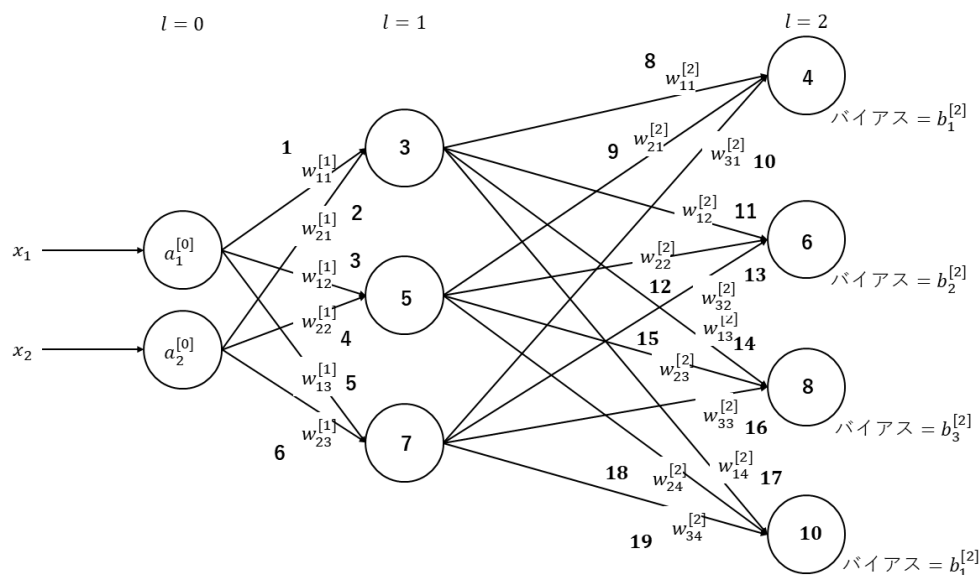


図 22 簡単なニューラルネットワークの例

- (2) 複数レイヤーを持つディープニューラルネットワークを構築し、MNIST データの推論を行った。手順を以下に示す。準備に図 23 のコードを実行した。 $\text{lambda } z: \text{np.maximum}(z, 0)$ は、 z を引数にとり z と 0 から最大の数を出す関数である。

```
1 import numpy as np
2 from keras.datasets import mnist
3 import matplotlib.pyplot as plt
4 rlu_gn=lambda Z: np.maximum(Z, 0)
5 from scipy.special import softmax
6 softmax_fn=lambda Z: softmax(Z, axis=1)
```

図 23 ニューラルネットワークを作る準備のためのコード

- ① **Layer** クラスを実装した。**Layer** クラスはニューラルネットワークの 1 層を実装するためのクラスである。図 24 に **Layer** クラスのコードを示す。

```
1 class Layer:
2     def __init__(self, prev_layer_size, layer_size, activation_fn):
3         np.random.seed(1)
4         self.W=np.random.randn(prev_layer_size, layer_size)/np.sqrt(prev_layer_size)
5         self.b=np.zeros(layer_size)
6         self.activation_fn=activation_fn
7     def forward(self, A_prev):
8         Z=np.dot(A_prev, self.W)+self.b
9         A=self.activation_fn(Z)
10        return A
```

図 24 Layer クラスのコード

- ② **Simple Classifier** クラスを実装した。**Simple Classifier** は **Layer** クラスを用いて、複数の層を持つニューラルネットワークを実装するために用いる。図 25 にコードを示す。

```
[8] ✓
1 class SimpleClassifier:
2     def __init__(self, input_layer_size, output_layer_size, hidden_layers_sizes, activation_fn=relu_fn):
3         layer_sizes=[input_layer_size,*hidden_layers_sizes]
4         self.layers=( [Layer(layer_sizes[i], layer_sizes[i+1], activation_fn)
5                         for i in range(len(layer_sizes)-1)] )
6         output_layer=Layer(layer_sizes[-1], output_layer_size, softmax_fn)
7         self.layers.append(output_layer)
8
9     def forward(self, A0):
10        A=A0
11        for layer in self.layers:
12            A=layer.forward(A)
13        Y_hat=A
14        return Y_hat
15
16    def predict(self, X):
17        Y_hat=self.forward(X)
18        return Y_hat
19
20    def evaluate_accuracy(self, X, Y):
21        predict=np.argmax(self.predict(X), axis=1)
22        actual=np.argmax(Y, axis=1)
23        num_corrects=len(np.where(predict==actual)[0])
24        accuracy=num_corrects/len(X)
25        return accuracy
```

図 25 SimpleClassifier のコード

- ③ MNIST データを取得し、それぞれのデータを 255 で割ることで正規化した。図 26 にコードを示す。

```
1 (x_train,y_train),(x_test,y_test)=mnist.load_data()
2 x_train,x_test=x_train/255.0,x_test/255.0
```

図 26 MNIST データの取得

- ④ x_train,y_train,x_test,y_test それぞれのデータの形状を調べた。
⑤ 図 27 のコードを実行し、1 個目のテストデータのラベルと画像を出力した。

```
1 print(y_test[0])
2 plt.imshow(x_test[0],cmap=plt.cm.gray)
```

図 27 MNIST データの取得

- ⑥ 4096 個目のテストデータのラベルと画像を出力した。
⑦ データのフラット化と One-Hot エンコーディングを図 28 のコードを用いて実装した。

```
1 x_train_flat,x_test_flat=x_train.reshape(-1,28*28),x_test.reshape(-1,28*28)
2 num_classes=10
3 y_train_ohe=np.eye(num_classes)[y_train]
4 y_test_ohe=np.eye(num_classes)[y_test]
```

図 28 データのフラット化と One-Hot エンコーディング

- ⑧ MNIST データを分類するためのモデルを図 29 のコードで実装した。

```
1 mnist_classifier=SimpleClassifier(x_test_flat.shape[1],num_classes,[64,32])
```

図 29 MNIST データを分類するためのモデル

- ⑨ 以下のコードで 2 つのテストデータの数字を推論し、分類した。図 30 にコードを示す。

```
1 y_hat=mnist_classifier.predict(x_test_flat[0:1])
2 print('推論結果:\n',y_hat[0])
3 print('分類結果:',np.argmax(y_hat[0]))
```

図 30 テストデータの数字の推論

⑩ 全テストデータ分類と精度の算出を図 31 のコードを用いて実施した。

```
1 accuracy=mnist_classifier.evaluate_accuracy(x_test_flat,y_test_ohv)
2 print('正解率:{:.2f}%'.format(accuracy*100))
```

図 31 全テストデータ分類と精度の算出

4. 予習課題

4.1 ‘こんにちは’を出力するコードの作成及び実行

‘こんにちは’を出力するコードを図 32 に示す。

[19] ✓ 0 秒 ▼	1 print('こんにちは') #print関数で こんにちは を出力する
	こんにちは

図 32 予習課題 1 のコードおよび実行結果

4.2 27 を 7 で割った余りを計算するコードの作成及び実行

27 を 7 で割った余りを計算するコードを図 33 に示す。

[20] ✓ 0 秒 ▼	1 27%7 #%を用いて余りを出力する
	6

図 33 予習課題 2 のコードおよび実行結果

4.3 1 から 1000 までの和を計算するコードの作成及び実行

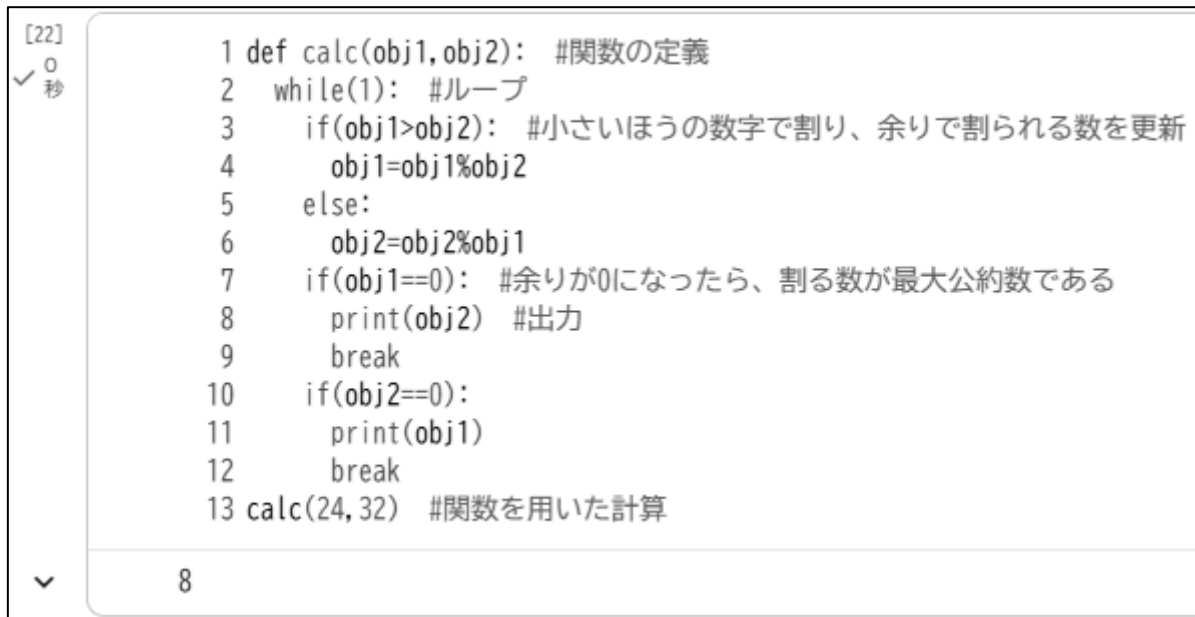
1 から 1000 までの和を計算するコードを図 34 に示す。

[21] ✓ 0 秒 ▼	1 a=0 2 for i in range(1001): #1000回ループ 3 a=a+i #ループ回数を足す 4 print(a) #結果の出力
	500500

図 34 予習課題 3 のコードおよび実行結果

4.4 予習課題 4

二つの自然数の最大の公約数を求める関数と、作成した関数を用いた 24 と 32 の計算結果を図 35 に示す。



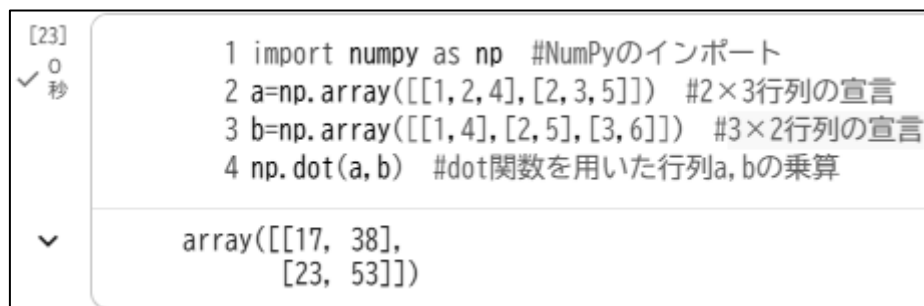
```
[22]
✓ 0 秒
1 def calc(obj1,obj2): #関数の定義
2     while(1): #ループ
3         if(obj1>obj2): #小さいほうの数字で割り、余りで割られる数を更新
4             obj1=obj1%obj2
5         else:
6             obj2=obj2%obj1
7         if(obj1==0): #余りが0になったら、割る数が最大公約数である
8             print(obj2) #出力
9             break
10        if(obj2==0):
11            print(obj1)
12            break
13 calc(24,32) #関数を用いた計算

8
```

図 35 予習課題 4 のコードおよび実行結果

4.5 予習課題 5

NumPy を用いた行列演算のコードおよび実行結果を図 36 に示す。



```
[23]
✓ 0 秒
1 import numpy as np #NumPyのインポート
2 a=np.array([[1,2,4],[2,3,5]]) #2×3行列の宣言
3 b=np.array([[1,4],[2,5],[3,6]]) #3×2行列の宣言
4 np.dot(a,b) #dot関数を用いた行列a,bの乗算

array([[17, 38],
       [23, 53]])
```

図 36 予習課題 5 のコードおよび実行結果

4.6 予習課題 6

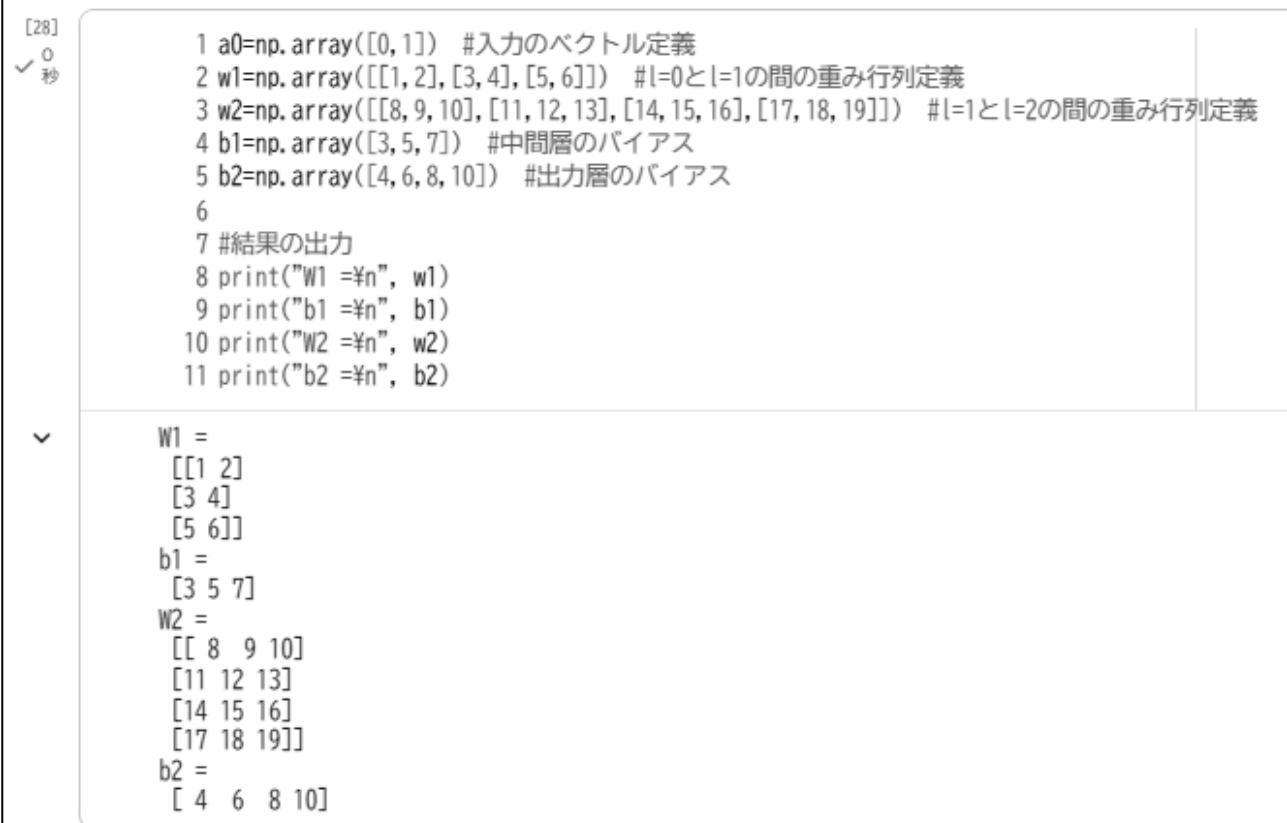
ステップ関数、シグモイド関数、ReLU 関数、ソフトマックス関数のコードと実行例を図 37 に示す。

[24] ✓ 0 秒	<pre>1 def step(x): #ステップ関数の定義 2 if x<0: #負の値の場合0を、正の値の場合1を返す 3 return 0 4 else: 5 return 1 6 print(step(10)) #xに10を入れてステップ関数を実行 7 print(step(-10)) #xに-10を入れてステップ関数を実行</pre>
▼	<pre>1 0</pre>
[25] ✓ 0 秒	<pre>1 import math as m #mathをインポート 2 def sig(x): #シグモイド関数の定義 3 return 1/(1+m.e**(-x)) #入力を計算して返す 4 print(sig(10)) #xに10を入れてシグモイド関数を実行 5 print(sig(-10)) #xに-10を入れてシグモイド関数を実行</pre>
▼	<pre>0.9999546021312976 4.539786870243442e-05</pre>
[26] ✓ 0 秒	<pre>1 def ReLU(x): #ReLU関数の定義 2 if x<0: #負の値の場合0を、正の値の場合そのまま返す 3 return 0 4 else: 5 return x 6 print(ReLU(10)) #xに10を入れてReLU関数を実行 7 print(ReLU(-10)) #xに-10を入れてReLU関数を実行</pre>
▼	<pre>10 0</pre>
[27] ✓ 0 秒	<pre>1 def softmax(x): #softmax関数の定義 2 ex=np.exp(x-np.max(x)) #入力された配列のそれぞれの要素に最大値を引き、指数関数を適用する 3 return ex/ex.sum(axis=0) #各要素を全体の合計値で割る 4 print(softmax([1, 5, 10])) #配列を入れてsoftmax関数を実行 5 print(softmax([-1, -5, -10]))</pre>
▼	<pre>[1.22568816e-04 6.69203059e-03 9.93185401e-01] [9.81894794e-01 1.79840305e-02 1.21175444e-04]</pre>

図 37 予習課題 6 のコードおよび実行結果

4.7 予習課題 7

重み行列およびバイアスベクトルを求めるコードと実行結果を図 38 に示す。



```
[28]
✓ 0 秒
1 a0=np.array([0,1]) #入力のベクトル定義
2 w1=np.array([[1,2],[3,4],[5,6]]) #l=0とl=1の間の重み行列定義
3 w2=np.array([[8,9,10],[11,12,13],[14,15,16],[17,18,19]]) #l=1とl=2の間の重み行列定義
4 b1=np.array([3,5,7]) #中間層のバイアス
5 b2=np.array([4,6,8,10]) #出力層のバイアス
6
7 #結果の出力
8 print("W1 =\n", w1)
9 print("b1 =\n", b1)
10 print("W2 =\n", w2)
11 print("b2 =\n", b2)
```

```
W1 =
[[1 2]
 [3 4]
 [5 6]]
b1 =
[3 5 7]
W2 =
[[ 8  9 10]
 [11 12 13]
 [14 15 16]
 [17 18 19]]
b2 =
[ 4  6  8 10]
```

図 38 予習課題 7 のコードおよび実行結果

4.8 予習課題 8

2.原理 2.8.Python の項において、コードおよび実行結果を図 14～21 に示す。

5. 使用器具

表 1 に、本実験の使用器具を示す。

表 1 使用器具

名称	製造会社	型式	製造番号	備品番号
PC	ASUS	F94J	TAN0CV06V67842B	—

6. 実験結果

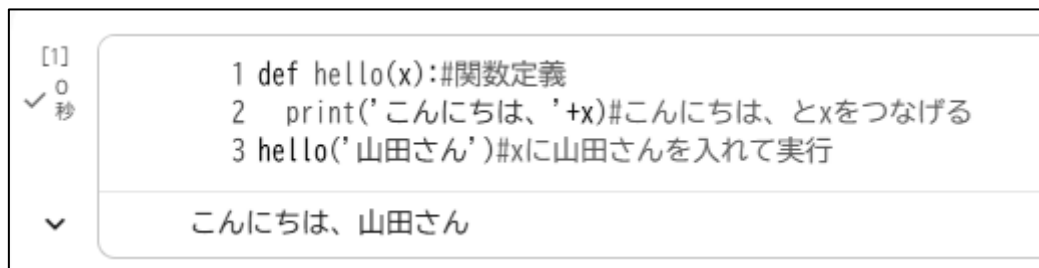
6.1 C1: ペアプログラミングによる Python コーディング

渡された課題を、コーディング担当と調査担当に分かれ、それぞれのコードについてペアプログラミングにより協力して解答した。

渡された課題と、解答したコードおよび実行結果を示す。

6.1.1

文字列を入力すると、”こんにちは、**さん”と表示する関数を書け。コードおよび実行結果を図 39 に示す。



```
[1]
✓ 0 秒
1 def hello(x):#関数定義
2   print(' こんにちは、'+x)#こんにちは、とxをつなげる
3 hello(' 山田さん')#xに山田さんを入れて実行

▼
    こんにちは、山田さん
```

図 39 課題 1 のコードおよび実行結果

6.1.2

リストから orange を表示するコードを書け。コードおよび実行結果を図 40 に示す。



```
[2]
✓ 0 秒
1 food=["apple","peach","orange"],["cabbage","carrot","potato"]#配列定義
2 print(food[0][2])#出力

▼
    orange
```

図 40 課題 2 のコードおよび実行結果

6.1.3

リストを 30 以下とそれ以外で分割するコードを書け。図 41 にコードおよび実行結果を示す。

[3]

```
1 lista=[20,56,70,13,62,49,97,29]#リスト
2 listb=[]
3 listc=[]
4 for i in lista:
5     if i>30:
6         listc.append(i)#30より大きい数をcに、それ以外をbに入れる
7     else:
8         listb.append(i)
9 print(listb[0:])
10 print(listc[0:])
```

▼

```
[20, 13, 29]
[56, 70, 62, 49, 97]
```

図 41 課題 3 のコードおよび実行結果

6.1.4

フィボナッチ数列を 231 まで生成するコードを書け。図 42 のコードおよび実行結果

[]

```
1 a=1
2 b=1
3 print(a)
4 print(b)
5 while(a+b<231):#231より小さい間、足し算と出力をループ
6     c=a
7     a=b
8     b=a+c
9     print(b)
```

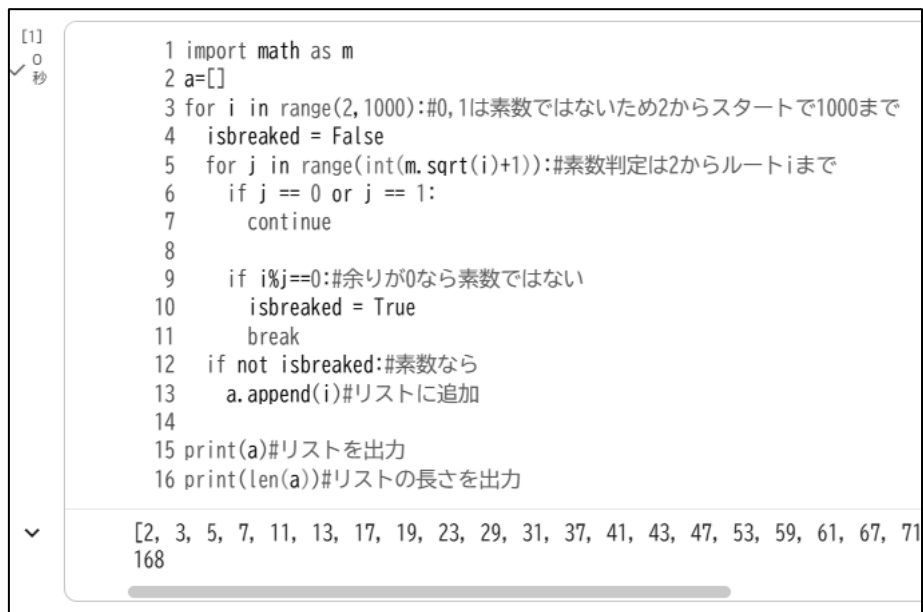
▼

```
1
1
2
3
5
8
13
21
34
55
89
144
```

図 42 課題 4 のコードおよび実行結果

6.1.5

1000 以下の素数を要素とするリストを表示するコードを記述せよ。また、1000 以下の素数の数を求めるコードを書け。図 43 にコードおよび実行結果を示す。



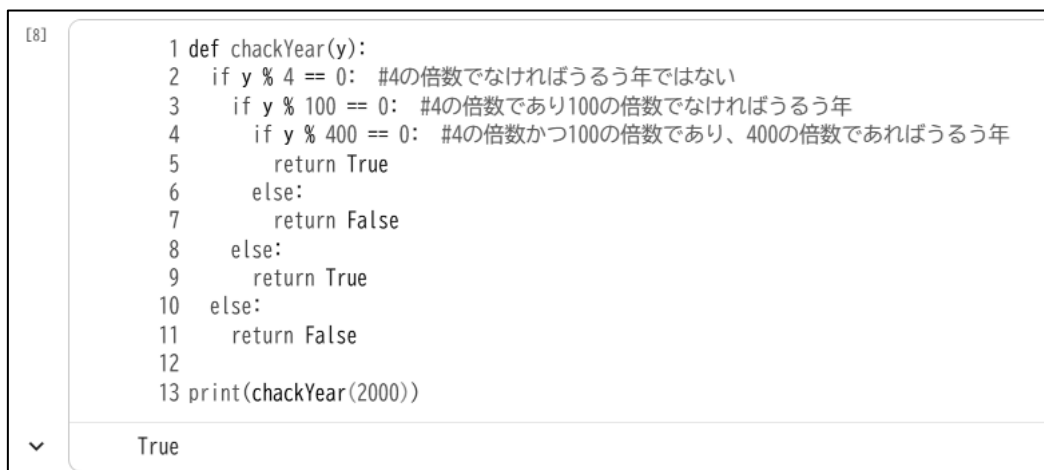
```
[1] 0 秒
✓
1 import math as m
2 a=[]
3 for i in range(2,1000):#0,1は素数ではないため2からスタートで1000まで
4     isbroken = False
5     for j in range(int(m.sqrt(i)+1)):#素数判定は2からルートiまで
6         if j == 0 or j == 1:
7             continue
8
9         if i%j==0:#余りが0なら素数ではない
10            isbroken = True
11            break
12     if not isbroken:#素数なら
13         a.append(i)#リストに追加
14
15 print(a)#リストを出力
16 print(len(a))#リストの長さを出力

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 187, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 473, 479, 487, 491, 499, 503, 509, 521, 523, 527, 539, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 623, 629, 631, 637, 641, 643, 647, 653, 659, 661, 667, 671, 673, 677, 683, 687, 691, 697, 701, 703, 709, 713, 719, 727, 733, 737, 739, 743, 749, 757, 761, 763, 769, 773, 779, 781, 787, 791, 793, 797, 803, 809, 811, 817, 821, 823, 827, 829, 833, 837, 839, 843, 847, 851, 853, 857, 859, 863, 867, 869, 871, 873, 877, 881, 883, 887, 893, 897, 901, 903, 907, 911, 913, 917, 919, 923, 927, 929, 931, 933, 937, 939, 941, 943, 947, 949, 953, 957, 959, 961, 963, 967, 969, 971, 973, 977, 979, 983, 987, 989, 991, 993, 997, 999]
168
```

図 43 課題 5 のコードおよび実行結果

6.1.6

入力した西暦がうるう年かどうか判別するプログラムを作成せよ。図 44 にコードおよび実行結果を示す。



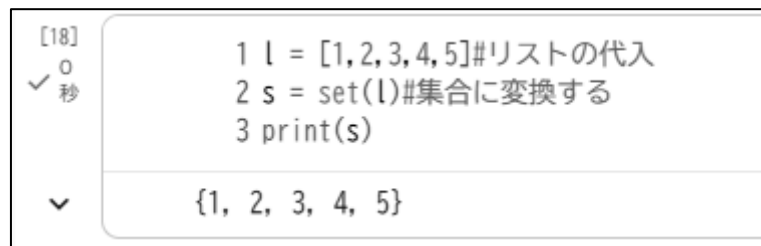
```
[8]
1 def chackYear(y):
2     if y % 4 == 0: #4の倍数でなければうるう年ではない
3         if y % 100 == 0: #4の倍数であり100の倍数でなければうるう年
4             if y % 400 == 0: #4の倍数かつ100の倍数であり、400の倍数であればうるう年
5                 return True
6             else:
7                 return False
8         else:
9             return True
10    else:
11        return False
12
13 print(chackYear(2000))

True
```

図 44 6 個目の課題のコードおよび実行結果

6.1.7

変数にリスト[1,2,3,4,5]を代入し、それを集合に変換するプログラムを作成せよ。図 45 にコードおよび実行結果を示す。



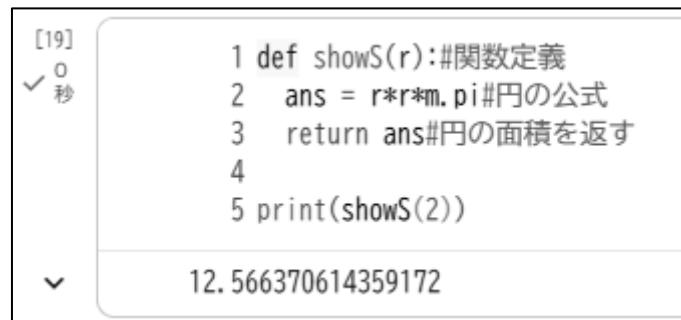
```
[18]
✓ 0 秒
1 l = [1, 2, 3, 4, 5] # リストの代入
2 s = set(l) # 集合に変換する
3 print(s)

{1, 2, 3, 4, 5}
```

図 45 7 個目の課題のコードおよび実行結果

6.1.8

半径を与えると、円の面積を返す関数を定義するプログラムを作成せよ。図 46 にコードおよび実行結果を図に示す。



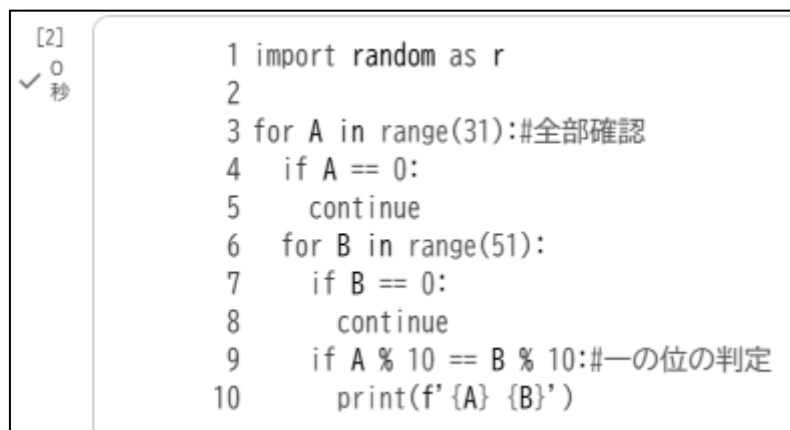
```
[19]
✓ 0 秒
1 def showS(r): # 関数定義
2     ans = r*r*m.pi # 円の公式
3     return ans # 円の面積を返す
4
5 print(showS(2))

12.566370614359172
```

図 46 8 個目の課題のコードおよび実行結果

6.1.9

1 から 30 の番号が付いた玉が入っている袋と、1 から 50 の番号が付いた玉が入っている袋がある。2 つの袋から 1 個ずつ玉を取ってきたとき、両方の玉の 1 の位が一致する組み合わせをすべて出力せよ。コードを図 47 に示す。



```
[2]
✓ 0 秒
1 import random as r
2
3 for A in range(31): # 全部確認
4     if A == 0:
5         continue
6     for B in range(51):
7         if B == 0:
8             continue
9         if A % 10 == B % 10: # 一の位の判定
10            print(f'{A} {B}')
```

図 47 9 個目の課題のコード

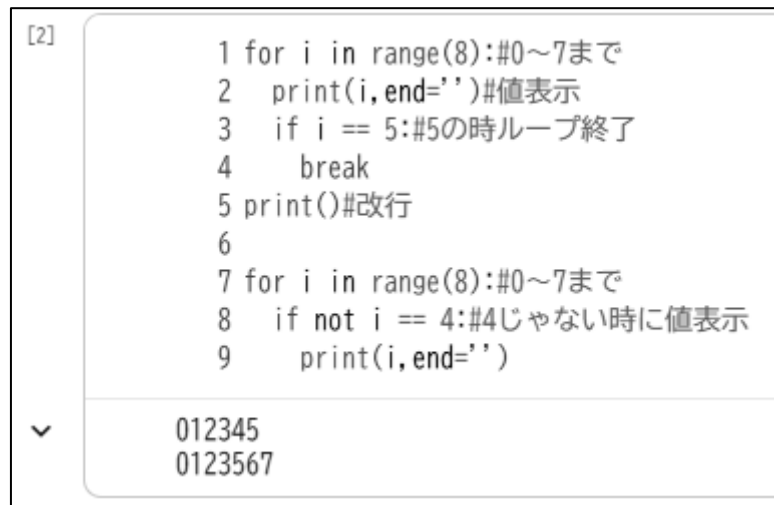
実行結果を図 48 に示す。

1 1	9 39	18 18	26 46
1 11	9 49	18 28	27 7
1 21	10 10	18 38	27 17
1 31	10 20	18 48	27 27
1 41	10 30	19 9	27 37
2 2	10 40	19 19	27 47
2 12	10 50	19 29	28 8
2 22	11 1	19 39	28 18
2 32	11 11	19 49	28 28
2 42	11 21	20 10	28 38
3 3	11 31	20 20	28 48
3 13	11 41	20 30	29 9
3 23	12 2	20 40	29 19
3 33	12 12	20 50	29 29
3 43	12 22	21 1	29 39
4 4	12 32	21 11	29 49
4 14	12 42	21 21	30 10
4 24	13 3	21 31	30 20
4 34	13 13	21 41	30 30
4 44	13 23	22 2	30 40
5 5	13 33	22 12	30 50
5 15	13 43	22 22	
5 25	14 4	22 32	
5 35	14 14	22 42	
5 45	14 24	23 3	
6 6	14 34	23 13	
6 16	14 44	23 23	
6 26	15 5	23 33	
6 36	15 15	23 43	
6 46	15 25	24 4	
7 7	15 35	24 14	
7 17	15 45	24 24	
7 27	16 6	24 34	
7 37	16 16	24 44	
7 47	16 26	25 5	
8 8	16 36	25 15	
8 18	16 46	25 25	
8 28	17 7	25 35	
8 38	17 17	25 45	
8 48	17 27	26 6	
9 9	17 37	26 16	
9 19	17 47	26 26	
9 29	18 8	26 36	

図 48 9 個目の課題の実行結果

6.1.10

0-7 の値を for 文を使用して一つずつ表示。ただし 5 になったらループを終了させよ。また、0-7 の値を一つずつ表示せよ。ただし 4 の場合は表示しない。コードおよび実行結果を図 49 に示す。



```
[2] 1 for i in range(8):#0~7まで
    2   print(i,end='')#値表示
    3   if i == 5:#5の時ループ終了
    4       break
    5   print()#改行
    6
    7 for i in range(8):#0~7まで
    8   if not i == 4:#4じゃない時に値表示
    9       print(i,end='')

012345
0123567
```

図 49 10 個目の課題のコードおよび実行結果

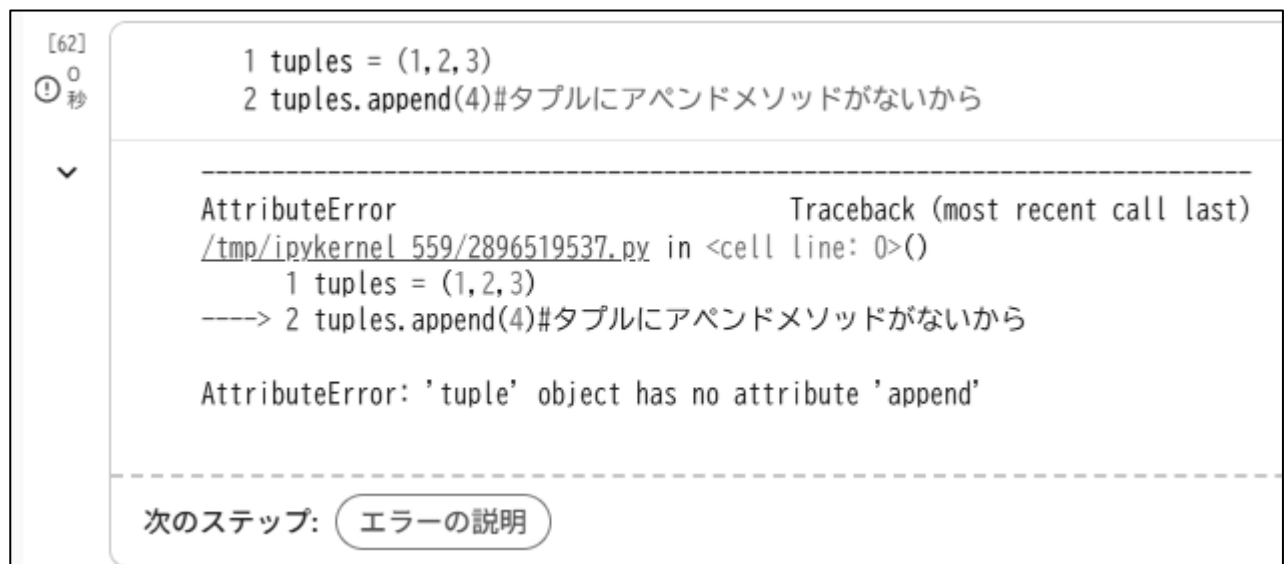
6.1.11

以下のコードを実行し、発生するエラーについて説明せよ。

```
tuples=(1,2,3)
```

```
tuples.append(4)
```

コードおよび実行結果を図 50 に示す。



```
[62] 1 tuples = (1, 2, 3)
    2 tuples.append(4)#タプルにアペンドメソッドがないから

AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_559/2896519537.py in <cell line: 0>()
      1 tuples = (1, 2, 3)
----> 2 tuples.append(4)#タプルにアペンドメソッドがないから

AttributeError: 'tuple' object has no attribute 'append'

次のステップ: エラーの説明
```

図 50 11 個目の課題のコードおよび実行結果

6.1.12

長さ L の棒を角棒の長さが全て正整数になるように 5 か所で切断して 6 本に分割するときに、分割の仕方が何通りあるかを求める関数を定義せよ。ただし、 L は $(6 \leq L \leq 100)$ を満たす整数とする。

例) $L=7$ のとき

[2,1,1,1,1,1][1,2,1,1,1,1][1,1,2,1,1,1][1,1,1,2,1,1][1,1,1,1,2,1][1,1,1,1,1,2]

の 6 通りとなる。コードおよび実行結果を図 51 に示す。

```
[42] 1 def debine(L):
      2     cnt = 0
      3     for l1 in range(L):#全部判定する
      4         for l2 in range(L):
      5             for l3 in range(L):
      6                 for l4 in range(L):
      7                     for l5 in range(L):
      8                         for l6 in range(L):
      9                             if l1 == 0 or l2 == 0 or l3 == 0 or l4 == 0 or l5 == 0 or l6 == 0:#0じゃない時
     10                                 continue
     11                             if l1 + l2 + l3 + l4 + l5 + l6 == L:#合計がLと同じなら
     12                                 cnt += 1#カウントする
     13     return cnt
     14 print(debine(6))
     15 print(debine(7))
```

1
6

図 51 12 個目の課題のコードおよび実行結果

6.1.13

numpy,matplotlib をインポートし、以後それぞれ np,plt という名前で参照できるようにせよ。コードおよび実行結果を図 52 に示す。

```
[43] 1 import numpy as np#NumPyをnpとしてインポート
      2 import matplotlib as plt#matplotlibをpltとしてインポート
```

図 52 13 個目の課題のコードおよび実行結果

6.1.14

10×10 の単位行列を `numpy` で作れ。コードおよび実行結果を図 53 に示す。

```
[44] 0秒
✓
1 X = np.eye(10)#no. eyeは単位行列を出力する
2 print(X)

[[1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]
```

図 53 14 個目の課題のコードおよび実行結果

6.1.15

以下を満たす $a_1, \dots, a_{k-1}$ を考える。

$$a_i \geq 0 (1 \leq i \leq k-1)$$

$$\sum_{i=1}^{k-1} a_i = 1$$

このとき、すべての $a_i (1 \leq i \leq k-1)$ に対して

$$y_i = \frac{\exp(a_i)}{\sum_{i=1}^{k-1} \exp(a_i)}$$

を計算する関数を実装せよ。また、 $a_1 = 0.1, a_2 = 0.3, a_3 = 0.6$ の場合の実行結果も示せ。コードおよび実行結果を図 54 に示す。

```
[45]
1 def Y(a):
2     ans = []
3     bunsu = 0
4     for i in a:#ここで分子を計算する
5         bunsu += m.e**i
6     for i in a:#ここで解を計算する
7         ans.append(m.e**i/bunsu)
8     return ans#答えを返す
9
10 A = [0.1, 0.3, 0.5]
11 B = Y(A)
12 print(B)

[0.2693074991777379, 0.3289329222889067, 0.4017595785333555]
```

図 54 15 個目の課題のコードおよび実行結果

6. 1. 16

`numpy` を使って、昨日今日明日の日付を出力せよ。コードおよび実行結果を図 55 に示す。

```
[45] 1 T = np.datetime64('now','D')#今日の日付を出力する
      2 T = str(T)
      3 t = T.split('-')#-で文字列を区切る
      4
      5 print(f'{t[0]}-{t[1]}-{int(t[2])-1}')#日付部分だけ+1
      6 print(T)
      7 print(f'{t[0]}-{t[1]}-{int(t[2])+1}')
      8
```

2026-05-8
2026-05-09
2026-05-10

図 55 課題 16 のコードおよび実行結果

6. 1. 17

要素が全て 0 で長さが 100 の numpy 配列を作れ。コードおよび実行結果を図 56 に示す。

[illegible]

図 56 課題 17 のコードおよび実行結果

6. 1. 18

要素を乱数で生成し、 $2 \times 2 \times 2$ の 3 次元行列を作れ。コードおよび実行結果を図 57 に示す。

[23] 0 秒	<pre>1 import random 2 3 #すべての場所にランダム入力する 4 A = np.array([[random.random() for _ in range(2)], [random.random() for _ in range(2)], [[random.random() for _ in range(2)], [random.random() for _ in range(2)]]]) 5 print(A)</pre>	
✓	<pre>[[[0.08294771 0.56026329] [0.69270737 0.43080094]] [[0.95065535 0.47216484] [0.15699694 0.05203736]]]</pre>	

図 57 課題 18 のコードおよび実行結果

6.1.19

初項 1、交差 2 の等差数列を 25 番目まで出力せよ。コードおよび実行結果を図 58 に示す。

```
[24]
1 ans = 1#初項
2 for i in range(25):#25番目まで2を加算して出力
3     ans += 2
4     print(f'{ans} ', end='')

3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51
```

図 58 課題 19 のコードおよび実行結果

6.1.20

3×3×3 配列を乱数で生成して、要素の最小値と最大値を出力せよ。ただし、要素はすべて 0～100 の整数で生成せよ。コードおよび実行結果を図 59 に示す。

```
[100]
0秒
✓
1 #ランダムな配列生成
2 A = [[[int(random.random() * 100) for _ in range(3)] for _ in range(3)] for _ in range(3)]
3 M,mi =0,999#最大値と最小値
4 for i in range(3):
5     for j in range(3):
6         for k in range(3):#全ての要素で最大値、最小値を比較し、更新
7             if A[i][j][k] > M: M = A[i][j][k]
8             if A[i][j][k] < mi: mi = A[i][j][k]
9 print(f'Max={M} min={mi}')

Max=95 min=3
```

図 59 課題 20 のコードおよび実行結果

6.1.21

8×8 の配列を作成し、0,1 で市松模様を出力せよ。コードおよび実行結果を図 60 に示す。

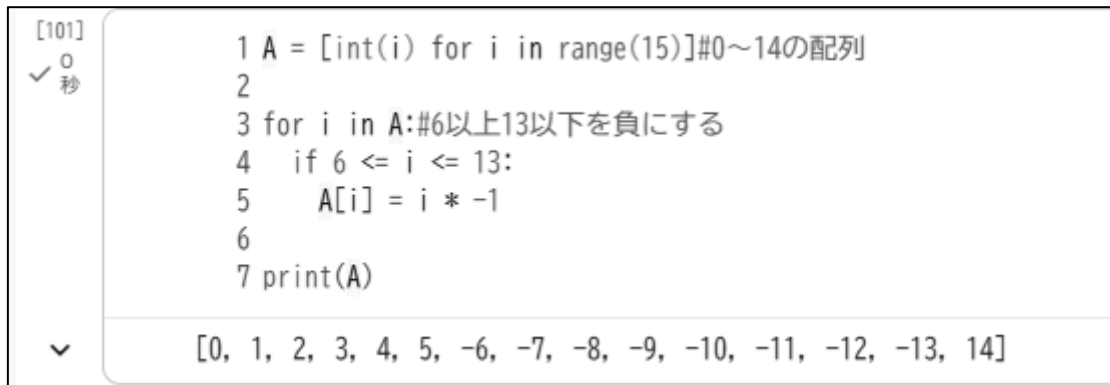
```
[26]
1 #要素が0の配列
2 A = [[0 for _ in range(8)] for _ in range(8)]
3
4 for a in range(8):
5     for b in range(8):
6         if (a + b) % 2 == 1:#奇数か偶数かで判定する
7             A[a][b] = 1#要素を1にする
8
9 print(A)

[[0, 1, 0, 1, 0, 1, 0, 1], [1, 0, 1, 0, 1, 0, 1, 0], [0, 1, 0, 1, 0, 1, 0, 1],
```

図 60 課題 21 のコードおよび実行結果

6.1.22

0～14 の要素を持つ一次元配列をまず作り、つぎにその配列の 6 以上 13 以下の値は負の値にせよ。
コードおよび実行結果を図 61 に示す。



The image shows a Jupyter Notebook cell. On the left, there is a status bar with a checkmark, the text "[101]", and "0 秒". The main area contains the following Python code:

```
1 A = [int(i) for i in range(15)]#0～14の配列
2
3 for i in A:#6以上13以下を負にする
4     if 6 <= i <= 13:
5         A[i] = i * -1
6
7 print(A)
```

Below the code, the execution result is displayed in a light blue box:

```
[0, 1, 2, 3, 4, 5, -6, -7, -8, -9, -10, -11, -12, -13, 14]
```

図 61 課題 22 のコードおよび実行結果

6.1.23

(a) 2次元リストの大きさを出力する処理を書き、任意の2次元リストの大きさを出力せよ。コードおよび実行結果を図 62 に示す。

(b) 二つの2次元リストの入力にたいして、行列積を計算する関数を実装せよ。コードおよび実行結果を図 63 に示す。

(c) 予習課題 5 における行列積を、実装した関数を用いて求め、予習課題での答えと一致することをたしかめよ。コードおよび実行結果を図 64 に示す。

```
[28]
✓ 0 秒
1 def len2(l):
2     A = len(l[0])#それぞれの長さを出力
3     B = len(l[1])
4     return [A,B]
5
6 Len = len2([[1,1],[1,1]])
7 print(f' {Len[0]} {Len[1]}')
```

2 2

図 62 課題 23(a)のコードおよび実行結果

```
[29]
✓ 0 秒
1 def seki(ListA, ListB):
2     A = np.array(ListA)
3     B = np.array(ListB)
4     return A @ B#* では計算できない
5
6 A = [[1,1],[1,1]]
7 B = [[1,1],[1,1]]
8 print(seki(A,B))
```

[[2 2]
[2 2]]

図 63 課題 23(b)のコードおよび実行結果

```
[30]
✓ 0 秒
1 A = [[1,2,4],[2,3,5]]
2 B = [[1,4],[2,5],[3,6]]
3 print(seki(A,B))
```

[[17 38]
[23 53]]

図 64 課題 23(c)のコードおよび実行結果

6. 1. 24

for 文を用いて、list=[3,8,10,11,142,20,2]における最大値とそのインデックスを出力せよ。コードおよび実行結果を図 65 に示す。

```
[31] 1 list = [3, 8, 10, 11, 142, 20, 2]
      2
      3 Max = 0
      4 Max_index = 0
      5 for i in range(len(list)):
      6     if list[i] > Max:#最大を判定する
      7         Max = list[i]
      8         Max_index = i#最大値のインデックス
      9
     10 print(f'Max = {Max} Max_index = {Max_index}')
```

Max = 142 Max_index = 4

図 65 課題 24 のコードおよび実行結果

6. 1. 25

次の numpy 配列を平坦化(1次元化)せよ。

ndarray : [[1,2],[3,4],[5,6]]

コードおよび実行結果を図 66 に示す。

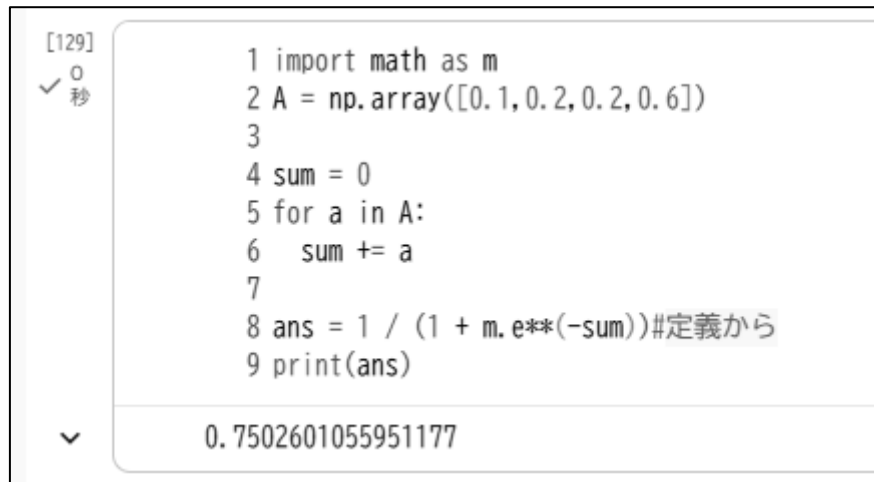
```
[32] 1 A = np.array([[1, 2], [3, 4], [5, 6]])
      2 A = A.flatten()#平坦化するメソッド
      3 print(A)
```

[1 2 3 4 5 6]

図 66 課題 25 のコードおよび実行結果

6. 1. 26

numpy を活用し、1 次元配列の入力の総和に対する、シグモイド関数の出力を行う関数を実装せよ（シグモイド関数を活性化関数とするパーセプトロンを実装せよ）また、作成した関数を用いて [0.1,0.2,0.2,0.6] を入力した場合の出力を求めよ。コードおよび実行結果を図 67 に示す。



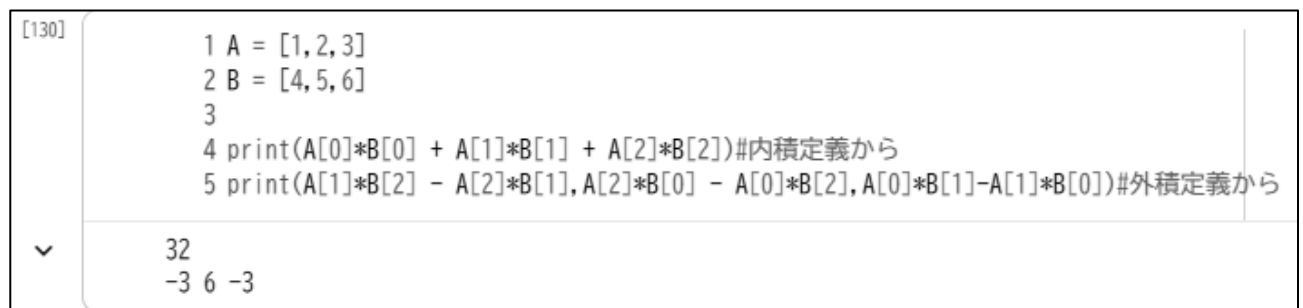
```
[129]
✓ 0 秒
1 import math as m
2 A = np.array([0.1, 0.2, 0.2, 0.6])
3
4 sum = 0
5 for a in A:
6     sum += a
7
8 ans = 1 / (1 + m.e**(-sum)) #定義から
9 print(ans)

0.7502601055951177
```

図 67 課題 26 のコードおよび実行結果

6. 1. 27

$\vec{a} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \vec{b} = \begin{pmatrix} 4 \\ 5 \\ 3 \end{pmatrix}$ としたとき、内積と外積を求めよ。コードおよび実行結果を図 68 に示す。



```
[130]
1 A = [1, 2, 3]
2 B = [4, 5, 6]
3
4 print(A[0]*B[0] + A[1]*B[1] + A[2]*B[2]) #内積定義から
5 print(A[1]*B[2] - A[2]*B[1], A[2]*B[0] - A[0]*B[2], A[0]*B[1]-A[1]*B[0]) #外積定義から

32
-3 6 -3
```

図 68 課題 27 のコードおよび実行結果

6.2 C2:Python によるニューラルネットワークの構築

6.2.1

図 26 のニューラルネットワークについて、入力値を x_1 , $x_2=(0,1)$ として NumPy を用いて実装した。コードおよび出力結果を図 69 に示す。

```
[16] 0秒
✓
1 a0=np.array([0,1])
2 w1=np.array([[1, 3, 5], [2, 4, 6]])
3 w2=np.array([[8, 11, 14, 17], [9, 12, 15, 18], [10, 13, 16, 19]])
4 b1=np.array([3, 5, 7])
5 b2=np.array([4, 6, 8, 10])
6
7 a1=a0@w1+b1
8 a2=a1@w2+b2
9
10 print(a1)
11 print(a2)
✓
[ 5  9 13]
[255 338 421 504]
```

図 69 NumPy を用いた図 26 のニューラルネットワークの実装

6.2.2

複数レイヤーを持つディープニューラルネットワークを構築し、MNIST データを用いて推論を行った。準備として作成したコードを図 70 に示す。

```
[23]
1 import numpy as np
2 from keras.datasets import mnist
3 import matplotlib.pyplot as plt
4 relu_fn=lambda Z: np.maximum(Z, 0)#Zと0で最大値を取るReLU関数
5 from scipy.special import softmax
6 softmax_fn=lambda Z: softmax(Z, axis=1)#ソフトマックス関数
```

図 70 ディープニューラルネットワーク構築の準備

Layer クラスのコードを図 71 に示す。

```
[24]
1 class Layer:#1層分の実装
2 #引数に1個前のレイヤーサイズ、現在のレイヤーサイズ、活性化関数を持つコンストラクタ定義
3 def __init__(self, prev_layer_size, layer_size, activation_fn):
4     np.random.seed(1)
5     self.W=np.random.randn(prev_layer_size, layer_size)/np.sqrt(prev_layer_size)#レイヤーの重みをランダムで生成
6     self.b=np.zeros(layer_size)#バイアスを0で生成
7     self.activation_fn=activation_fn#活性化関数に入力された活性化関数を入れる
8 def forward(self, A_prev):
9     Z=np.dot(A_prev, self.W)+self.b#1 個前の出力に重みを掛け、バイアスを足す
10    A=self.activation_fn(Z)#活性化関数に入れる
11    return A#Aを返す
```

図 71 Layer クラスのコード

SimpleClassifier クラスのコードを図 72 に示す。

```
[25] 1 class SimpleClassifier:#複数層の実装
2 #入力、出力、隠れ層のサイズ、活性化関数をReLU関数で定義
3 def __init__(self, input_layer_size, output_layer_size, hidden_layers_sizes, activation_fn=relu_fn):
4     layer_sizes=[input_layer_size,*hidden_layers_sizes]#レイヤーサイズを入力と隠れ層のリストで定義
5     self.layers=(Layer(layer_sizes[i], layer_sizes[i+1], activation_fn)
6         for i in range(len(layer_sizes)-1))#self.layersに複数層作成し格納
7     output_layer=Layer(layer_sizes[-1], output_layer_size, softmax_fn)#出力層をソフトマックス関数で定義
8     self.layers.append(output_layer)#layersに出力層を追加
9
10 def forward(self, A0):#内部計算用
11     A=A0
12     for layer in self.layers:#層の数だけループ
13         A=layer.forward(A)#Layerクラスのforward関数を実行して層の処理
14     Y_hat=A#Y_hatに実行結果を格納
15     return Y_hat
16
17 def predict(self, X):#外部からの呼び出し用
18     Y_hat=self.forward(X)#y_hatにforward関数の実行結果のy_hatを格納
19     return Y_hat#推論結果を返す
20
21 def evaluate_accuracy(self, X, Y):
22     predict=np.argmax(self.predict(X), axis=1)#推論結果の最大値を取り出す
23     actual=np.argmax(Y, axis=1)#正解の値
24     num_corrects=len(np.where(predict==actual)[0])#正解数をnum_correctsに入れる
25     accuracy=num_corrects/len(X)#正答率計算
26     return accuracy
```

図 72 SimpleClassifier のコード

MNIST データを取得するためのコードを図 73 に示す。

```
[26] ✓ 1 (x_train, y_train), (x_test, y_test)=mnist.load_data()
2 x_train, x_test=x_train/255.0, x_test/255.0
```

図 73 MNIST データのコード

最初のテストデータのラベルと画像の出力を行った。コードおよび実行結果を図 74 に示す。

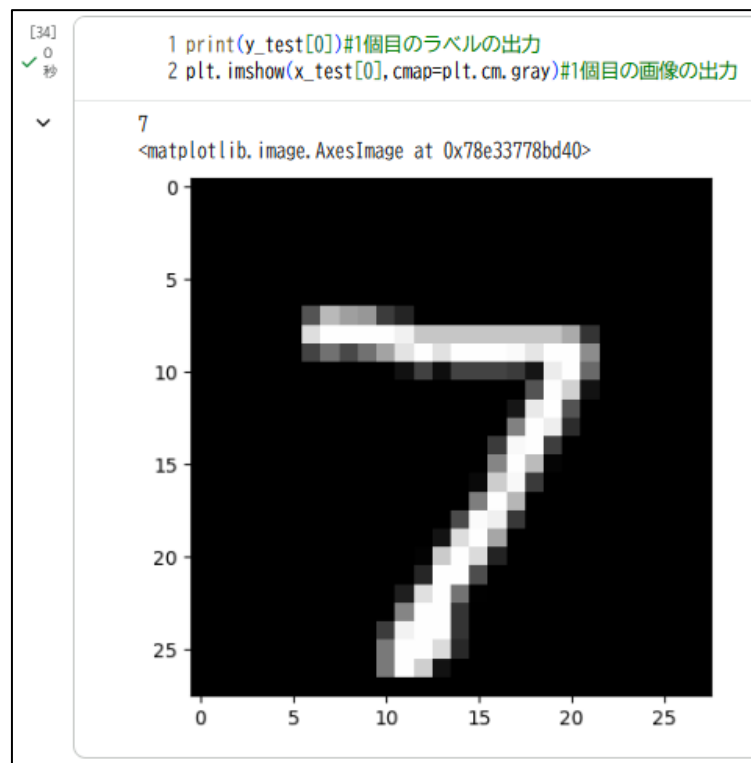


図 74 最初のテストデータのラベルと画像出力のコードおよび実行結果

4096 個目のテストデータのラベルと画像の出力を行った。コードおよび実行結果を図 75 に示す。

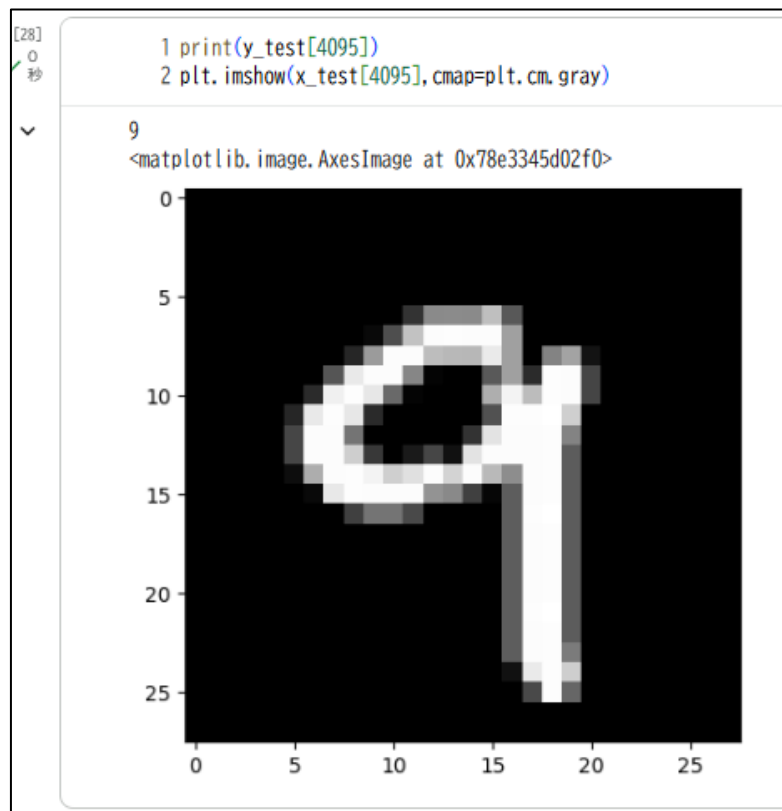


図 75 4096 個目のテストデータのラベルと画像のコードおよび実行結果

データの平坦化及び One-Hot エンコーディングのコードを図 76 に示す。

```
[35] 1 #平坦化
      2 x_train_flat,x_test_flat=x_train.reshape(-1,28*28),x_test.reshape(-1,28*28)
      3 num_classes=10
      4 #One-Hotエンコーディングに変換
      5 y_train_ohe=np.eye(num_classes)[y_train]
      6 y_test_ohe=np.eye(num_classes)[y_test]
```

図 76 平坦化および One-Hot エンコーディングのコード

MNIST データの分類を行うための、ニューラルネットワークのモデル構築を行った。コードを図 77 に示す。

```
[30] 1 #MNISTデータの分類 平坦化したデータの列数を入力層、分類の数を出力層、隠れ層の構成を64個と32個に定義
      2 mnist_classifier=SimpleClassifier(x_test_flat.shape[1],num_classes,[64,32])
```

図 77 MNIST データの分類を行うモデルのコード

MNIST の 1 個目のテストデータに対する推論を行った。コードおよび実行結果を図 78 に示す。

```
[10] 1 #1個目の平坦化したテストデータを推論する
✓ 0 2 y_hat=mnist_classifier.predict(x_test_flat[0:1])#
秒 3 print('推論結果:¥n',y_hat[0])
      4 print('分類結果:',np.argmax(y_hat[0]))

▼ 推論結果:
   [0.10292778 0.06061532 0.1338951  0.10314735 0.10864224 0.09773565
   0.11141222 0.08672163 0.09830012 0.09660259]
   分類結果: 2
```

図 78 1 個目のテストデータの推論

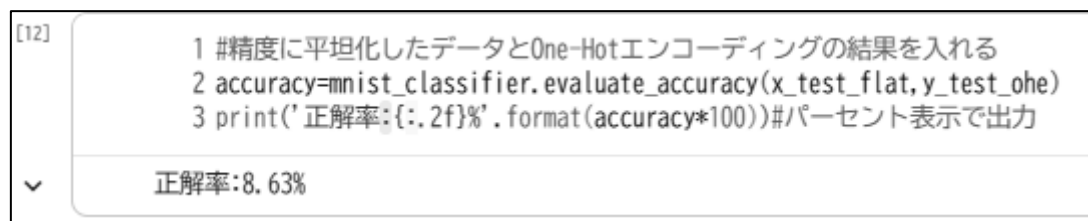
4096 個目のテストデータに対する推論を行った。コードおよび実行結果を図 79 に示す。

```
[13] 1 #4096個目の平坦化したテストデータを推論する
✓ 0 2 y_hat=mnist_classifier.predict(x_test_flat[0:4096])
秒 3 print('推論結果:¥n',y_hat[4095])
      4 print('分類結果:',np.argmax(y_hat[4095]))

▼ 推論結果:
   [0.09696259 0.06516943 0.14115156 0.1230866  0.10615438 0.09266565
   0.10382957 0.08291599 0.10077713 0.0872871 ]
   分類結果: 2
```

図 79 4096 個目のテストデータの推論

すべてのテストデータの分類と精度の算出を行った。コードおよび実行結果を図 80 に示す。



```
[12]: 1 #精度に平坦化したデータとOne-Hotエンコーディングの結果を入れる
      2 accuracy=mnist_classifier.evaluate_accuracy(x_test_flat,y_test_oh)
      3 print(' 正解率:{:.2f}%'.format(accuracy*100))#パーセント表示で出力

      正解率:8.63%
```

図 80 分類と精度のコードおよび実行結果

7. 考察

7.1 考察課題

7.1.1 ペアプログラミング

ペアプログラミングの教育、学習面と作業効率面の二点について効果を考察する。

教育、学習面では、1つのコードに二人で取り組むため、コードの内容や意図について共通認識を形成しながら作業を行うことができる。理解の相互補完が可能であるが、コードの理解など学習に意識を向けた場合、作業が遅くなると考えられるため注意が必要である。

作業効率の面では、ペアプログラミングによりコードの記述と調査に役割を分担することで1つのコードに対する作業の効率化を図ることができる。また、コードの確認を二人で行うことでスペルミスや構文ミスなどの間違いに気づくことができ、エラーが減少すると考えられる。しかし、役割分担を最も効果的に行うためには、ペアプログラミングの協力者全員がコードの理解をできている必要がある。それぞれの記述や構文方法に癖がある場合には理解に余分な時間を要することから、効果が下がるだろう。同様に、コードや関数、構文などの理解に差がある場合は協力者の役割がスペルチェックのような簡単なものに限られてしまうことから、効果が下がると考える。記述するコードがあまりにも短い場合や簡単な場合にも、協力者の役割が限られるため、作業効率の面では効果が下がる。反対に、コードが長く、難しい場合には作業時間が長くなり、二人で作業を行うことによる集中力の継続や、複雑な構造を複数の視点で見ることによる解決の早期化などの効果が高いと考える。

これらのことから、ペアプログラミングがうまくいくための要素として、二人の習熟度が近いこと、問題の難易度が高く、追加の調査が必要な程度であること、コーディングスタイルが統一されていることがあげられる。

7.1.2 MNIST データ

図 73 の `x_train` は学習用の画像データで、60000 枚である。`y_train` は画像データの各画像に対応する正解ラベルで、配列に格納されている。`x_test` はテスト用の画像データで、10000 枚である。`y_test` はテスト用の画像データの各画像に対応する正解ラベルである。

7.1.3 ディープニューラルネットワーク

C2 実験で使ったディープニューラルネットワークを図 81 に示す。

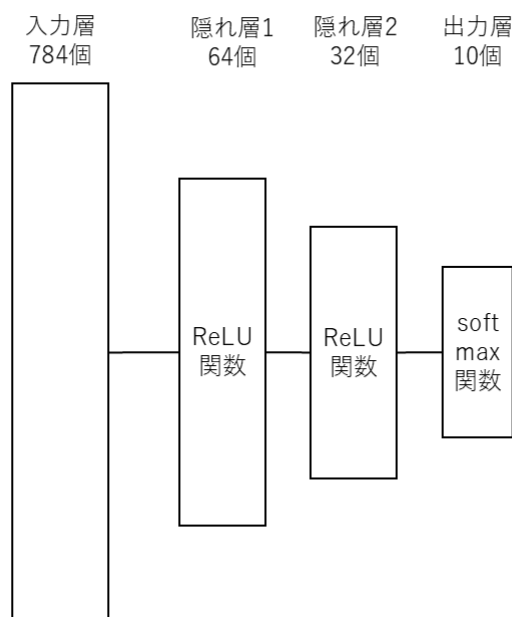


図 81 C2 実験で使ったディープニューラルネットワーク

28×28 ピクセルの入力画像を 784 個のノードに入力し、活性化関数に ReLU 関数を用いている 2 つの隠れ層を通した、出力層で 0～9 の 10 個の出力となる。出力層では活性化関数として softmax 関数が使用されているため、出力の合計は 1 となる。

7.1.4 One-Hot エンコーディング

図 76 より、1 行目は `x_train` と `x_test` を `reshape` 関数により画像枚数分の 60000 行、784 列の配列に変換している。画像データは 28×28 ピクセルであり、行を指定する部分が -1 となっているのは自動で行数を定義するためである。-1 としておくことによって、画像枚数を変更した場合でのエラーを予防でき、行数部分のコードの変更が不要となる。2 行目は、`numpy` の `eye` 関数を用いて出力の要素数の単位行列を作成し、`y_train` の行数のみを取り出すことで正解ラベルの One-Hot エンコーディングを行っている。One-Hot エンコーディングとは該当する箇所のみを 1 で、他を 0 で表現する形式に変換することである。

7.1.5 推論結果と分類結果の関わり

図 78 より、1 行目は 1 個目のテストデータをディープニューラルネットワークを通した出力結果を `y_hat` に格納している。`y_hat` は推論結果であり、出力層で softmax 関数に通しているため合計が 1 の確率の形式で表現されている。そのため、3 行目で最大値の要素を取り出し最もその要素である確率が高いという推論結果を基に分類結果が得られる。

7.1.6 推論結果と分類結果の精度

図 80 より、推論結果は 0~9 の各数字のニューラルネットワークからの出力である。各出力の値がそれぞれの数字への推論の確率であり、最も値が大きい数字が分類結果として出力される。今回の分類結果の精度は、出力結果より 8.63%であった。本実験において結果は 0~9 の 10 種類の数字であり、無作為に選んだ場合でも正解率の期待値は 10%となる。今回の結果は 10%よりも低いため、精度は低いと判断できる。使用したコードでは誤差の逆伝搬が実装されておらず、間違っただけであっても損失の勾配が出力側から入力側へ伝わらない。つまり、学習が一切行われず重みやバイアスなどのパラメータが変化しないため、解答も変化しない。よって、パラメータにランダムで初期化された重みが利用され続けたため、無作為よりも精度が低くなる結果となった。

7.1.7 確率的勾配降下法

確率的勾配降下法による学習コードを図 82 に示す。

```
1 relu_prime=lambda Z: (Z > 0).astype(float)
2 def softmax_prime(Z, Y_hat):
3     # For softmax with cross-entropy, the derivative dL/dZ is simply Y_hat - Y (true labels)
4     # However, for a generic softmax_prime, we return the Jacobian matrix or simpler case
5     # Since we'll use it with cross-entropy, we'll handle the combined derivative in SimpleClassifier.backward
6     # For now, let's keep a placeholder if needed generally, but it's often implicitly handled.
7     # For the purpose of backpropagation with cross-entropy, we only need the output delta (Y_hat - Y)
8     # so this function might not be directly used, but conceptually this is where it would be.
9     return Y_hat * (1 - Y_hat) # Placeholder, not strictly correct for chain rule dZ for softmax
```

図 82 確率的勾配降下法のコード 1

```

1 class Layer:
2     def __init__(self, prev_layer_size, layer_size, activation_fn, activation_prime_fn=None):
3         np.random.seed(1)
4         self.W=np.random.randn(prev_layer_size, layer_size)/np.sqrt(prev_layer_size) #ランダムな重み
5         self.b=np.zeros(layer_size) #バイアス 0
6         self.activation_fn=activation_fn
7         self.activation_prime_fn=activation_prime_fn # Derivative of activation function
8         self.A_prev = None # Stores input from previous layer for backprop
9         self.Z = None      # Stores pre-activation value for backprop
10
11     def forward(self, A_prev):
12         self.A_prev = A_prev
13         self.Z = np.dot(A_prev, self.W)+self.b
14         A=self.activation_fn(self.Z) #活性化関数
15         return A
16
17     def backward(self, dA, lr):
18         # dA: gradient of loss with respect to the activation of this layer
19
20         # If it's the output layer and using softmax with cross-entropy, dA is already dL/dZ for this layer
21         # Otherwise, apply activation derivative
22         if self.activation_prime_fn: # Only for hidden layers using relu_prime
23             dZ = dA * self.activation_prime_fn(self.Z)
24         else: # Output layer with softmax+cross-entropy, dA is effectively dL/dZ
25             dZ = dA
26
27         dW = np.dot(self.A_prev.T, dZ)
28         db = np.sum(dZ, axis=0)
29         dA_prev = np.dot(dZ, self.W.T)
30
31         # Update parameters
32         self.W -= lr * dW
33         self.b -= lr * db
34
35         return dA_prev # Gradient for the previous layer

```

図 83 確率的勾配降下法のコード2

```

1 class SimpleClassifier:
2     def __init__(self, input_layer_size, output_layer_size, hidden_layers_sizes, activation_fn=relu_fn, activation_prime=relu_prime):
3         layer_sizes=[input_layer_size,*hidden_layers_sizes]
4         self.layers=([Layer(layer_sizes[i], layer_sizes[i+1], activation_fn, activation_prime)
5             for i in range(len(layer_sizes)-1)])
6         output_layer=Layer(layer_sizes[-1], output_layer_size, softmax_fn) # Output layer uses softmax
7         self.layers.append(output_layer)
8
9     def forward(self, A0):
10         A=A0
11         for layer in self.layers:
12             A=layer.forward(A)
13         Y_hat=A
14         return Y_hat
15
16     def predict(self, X):
17         Y_hat=self.forward(X)
18         return Y_hat
19
20     def evaluate_accuracy(self, X, Y):
21         predict=np.argmax(self.predict(X), axis=1)
22         actual=np.argmax(Y, axis=1)
23         num_corrects=len(np.where(predict==actual)[0]) #推論の正解数
24         accuracy=num_corrects/len(X) #推論全体で割る
25         return accuracy #確率を返す
26
27     def backward(self, Y_hat, Y, lr):
28         # Calculate the gradient for the output layer (Softmax + Cross-Entropy)
29         # dL/dZ_L = Y_hat - Y (for softmax with cross-entropy loss)
30         dA = (Y_hat - Y) / Y.shape[0] # Average gradient over the batch
31
32         # Propagate gradients backward through the layers
33         for layer in reversed(self.layers):
34             dA = layer.backward(dA, lr)

```

図 84 確率的勾配降下法のコード 3

```

1 def cross_entropy_loss(Y_hat, Y):
2     # Y_hat: predicted probabilities (N, C) where N is batch size, C is num_classes
3     # Y: true labels (one-hot encoded) (N, C)
4     m = Y.shape[0] # Number of samples
5     # To avoid log(0), clip probabilities to a small epsilon
6     epsilon = 1e-12
7     Y_hat_clipped = np.clip(Y_hat, epsilon, 1. - epsilon)
8     loss = -np.sum(Y * np.log(Y_hat_clipped)) / m
9     return loss

```

図 85 確率的勾配降下法のコード 4

実行結果を図 86 に示す。

Epoch 1,	Loss: 0.2128,	Accuracy:93.49%
Epoch 2,	Loss: 0.3152,	Accuracy:93.55%
Epoch 3,	Loss: 0.0472,	Accuracy:96.13%
Epoch 4,	Loss: 0.3040,	Accuracy:96.66%
Epoch 5,	Loss: 0.0621,	Accuracy:96.83%
Epoch 6,	Loss: 0.0238,	Accuracy:97.06%
Epoch 7,	Loss: 0.0947,	Accuracy:96.74%
Epoch 8,	Loss: 0.0382,	Accuracy:97.24%
Epoch 9,	Loss: 0.1504,	Accuracy:96.28%
Epoch 10,	Loss: 0.0359,	Accuracy:96.79%
Epoch 11,	Loss: 0.0032,	Accuracy:97.39%
Epoch 12,	Loss: 0.0090,	Accuracy:97.38%
Epoch 13,	Loss: 0.0094,	Accuracy:97.18%
Epoch 14,	Loss: 0.0078,	Accuracy:97.36%
Epoch 15,	Loss: 0.0757,	Accuracy:96.99%
Epoch 16,	Loss: 0.0079,	Accuracy:97.51%
Epoch 17,	Loss: 0.0005,	Accuracy:97.62%
Epoch 18,	Loss: 0.0077,	Accuracy:97.21%
Epoch 19,	Loss: 0.0720,	Accuracy:95.60%
Epoch 20,	Loss: 0.0039,	Accuracy:97.45%

図 86 確率的勾配降下法の実行結果

図 82 のコードでは、活性化関数の定義を行っている。2 行目の `softmax` 関数の実装はテスト用の実装であり、実際には `softmax` 関数の実装ではないが、損失関数を利用して逆伝播させているため、 $y_hat*(1-y_hat)$ と簡単化できる。

図 83 のコードでは、`backward` メソッドが追加されたことで逆伝播の仕組みができている。`lr` の値を変えることで、学習率を変更して勾配をどの程度反映させるかを調整できる。

図 84 のコードでは、勾配である `dA` が追加されており、ミニバッチの誤差の平均を勾配として代入している。`dA` を用いて `backward` 関数を呼び出すことで逆伝播が行われ、学習により重みのパラメータが更新される。

図 85 のコードでは、損失関数の定義を行っている。交差エントロピー誤差が返り値となる。

図 86 より損失を逆伝播し、学習することで精度が上がっている。精度は最大で 97.62% となっており、95% 前後で安定しているため確率的勾配降下法は上手く機能している。エポック 6 以降精度は上昇と下降を繰り返しているため、6 回程度の学習で収束している。収束後にも精度は大きく変化していないため、局所的な適合解とはなっていないと考えられ、適切な学習が行われている。

7.2 実験考察

7.2.1 C1 実験

C1 実験では GoogleColaboratory を用いたペアプログラミングによって、提示された課題を Python で実装した。図 39~68 より、各課題のコードをペアプログラミングにより記述担当と、調査、エラー確認担当に作業を分担して効率的に課題を進めることができた。今回の実験においては調査、エラー確認を担当した。ペアプログラミングの効果としては、スペルミスの発見や構造などの着想の共有、調査の分担などにより、効率が上がった。今回の C1 実験により、Python のコーディング技術に加えてペアプログラミングの効果と実践方法を学ぶことができたと考える。

7.2.2 C2 実験

図 69~80 より、C2 実験では NumPy を用いたニューラルネットワークの実装を行った。その後 MNIST データセットを用いたディープニューラルネットワークの画像分類の実装を行った。ニューラルネットワークでは、重みとバイアスを用いた行列演算を行うことで入力から出力までの計算をコードで実装することができた。

MNIST データを用いたディープニューラルネットワークでは、画像分類の推論において逆伝播を行わないモデルは学習せず、精度が向上しないことを確認した。そのため、精度を向上するには学習を行うための逆伝播メソッドが必要である。また、softmax 関数と One-Hot エンコーディングを組み合わせることで、推論結果から分類を行い、正誤判定と精度の出力を実装することができた。

8. 参考文献

- [1] ディープラーニングがわかる数学入門, 涌井 良幸, 涌井 貞美, 技術評論社, 2017
- [2] ゼロから作る Deep Learning, 斎藤 康数, オライリー・ジャパン, 2016